ONSCREEN

Install & Administration Guide

Self-hosted media server · version 2.6

Audience: people who install, configure and run an OnScreen server.

Covers: the Go server and its transcode workers, PostgreSQL and Valkey, the web admin pages, and the settings the TV, phone, Roku, Xbox and desktop apps depend on.

Written: October 2026, against the v2.6 release candidate. Where a setting is described as “new in v2.6” it does not exist in v2.5.x.

Screens in this guide come from a development server; names and addresses in them are placeholders.

Contents

1 Introduction

1.1 What OnScreen is
1.2 Architecture at a glance
1.3 System requirements
1.4 Supported hardware encoders
1.5 Client apps

2 Installation

2.1 Choosing an installation path
2.2 Docker Compose (CPU image)
2.3 Docker with an NVIDIA GPU
2.4 Windows installer and portable zip
2.5 Linux portable tarball
2.6 Building from source (bare metal)
2.7 First-run setup

3 Configuration reference

3.1 How configuration works
3.2 Environment variables
3.3 Secrets and key rotation
3.4 Database and migrations

4 Networking and remote access

4.1 Ports
4.2 Reverse proxy
4.3 TRUSTED_PROXIES and client addresses
4.4 HTTPS
4.5 Cloudflare Tunnel and split segment access
4.6 Automatic router port mapping
4.7 The home network and the internet limit
4.8 LAN discovery and CORS

5 Libraries

5.1 Library types
5.2 Creating and configuring a library
5.3 Folder and file naming
5.4 Scanning
5.5 Metadata agents and API keys
5.6 Language and episode order
5.7 NFO files
5.8 Editing metadata, Fix Match and labels
5.9 Collections
5.10 Library health and maintenance

6 Transcoding and playback

6.1 How the play decision works
6.2 The Transcode settings page
6.3 Encoders and tuning
6.4 HDR tone mapping
6.5 The worker fleet
6.6 Limits: quality, bitrate and streams
6.7 Optimized versions and converted downloads
6.8 Subtitles
6.9 Audio passthrough on remux
6.10 Dolby Vision

7 Users and security

7.1 Users and invites
7.2 Managed profiles, PINs and rating limits
7.3 Per-user restrictions
7.4 Signed-in devices
7.5 Two-factor authentication
7.6 Single sign-on (OIDC, SAML, LDAP)
7.7 Personal API keys
7.8 Rate limits
7.9 Audit log
7.10 Security checklist

8 Live TV and DVR

8.1 Tuners and live broadcasts
8.2 Program guide (EPG)
8.3 Recordings and post-processing
8.4 Commercial detection

9 Integrations

9.1 Radarr, Sonarr and requests
9.2 Scrobbling
9.3 OPDS book catalog
9.4 M3U playlists
9.5 Webhooks and notification agents
9.6 Plugins (MCP)
9.7 Import from Plex, Jellyfin or Emby
9.8 Email and the digest
9.9 Object storage

10 Tasks and maintenance

10.1 Scheduled tasks
10.2 Backup and restore
10.3 Upgrading from v2.5 to v2.6
10.4 Monitoring, metrics and logs
10.5 High availability (optional)

11 Troubleshooting

12 Appendix

12.1 Ports
12.2 File and directory locations
12.3 Command-line tools
12.4 API documentation

1 Introduction

1.1 What OnScreen is

OnScreen is an open-source, self-hosted media server. It scans your movies, TV shows, anime, music, audiobooks, books and comics, photos, home videos and podcasts, fetches their metadata and artwork, and streams them to a web app and to native apps on TVs, phones, Roku, Xbox and desktop computers. Live TV, DVR, hardware transcoding and single sign-on are part of the core server; there is no paid tier.

The server is licensed AGPLv3. It is built on PostgreSQL rather than an embedded database, ships as a single Go binary with the web app embedded, and keeps watch state as an append-only event log that a database trigger rolls up into each user's progress.

1.2 Architecture at a glance

Clients
Web app (served by the server) · Android TV / Fire TV · Android phone · Samsung Tizen · LG webOS · Roku · Xbox · desktop app · `onscreen-cli`

OnScreen server
`server` binary: REST API (`/api/v1`), web UI, TV web app at `/tvapp/` , scanner, scheduler, Live TV/DVR, RTMP ingest, and an *embedded* transcode worker.

Transcode workers
The embedded worker, plus optional standalone `worker` processes on other machines. They pick jobs off a shared queue and serve HLS segments the server proxies.

FFmpeg
Used by the server and workers for probing (`ffprobe`), remuxing, transcoding, tone mapping, thumbnails and subtitle OCR (with Tesseract).

PostgreSQL 16+
All persistent data: libraries, items, users, watch events, settings (secrets encrypted with `SECRET_KEY`).

Valkey / Redis 7+
Sessions, the transcode job queue and session index, rate limits, caches, and the leader lock for singleton work.

Storage
Your media folders (local disk, or S3-compatible object storage), and the cache directory (`CACHE_PATH`) for artwork, thumbnails, converted downloads and optimized versions.

The API tier is stateless: session state lives in Valkey, so several server instances can run behind a load balancer. Everything beyond a single node (Valkey Sentinel, PostgreSQL failover, object storage, a CDN, multi-site DR) is optional and off by default; see 10.5.

1.3 System requirements

Component	Version	Notes
PostgreSQL	16 or newer	The first migration creates the `pg_trgm`, `pgcrypto` and `unaccent` extensions, which ship with PostgreSQL.
Valkey or Redis	7 or newer	Sessions, job queue, rate limiting.
FFmpeg (+ ffprobe)	latest stable	Bundled in the Docker images and the Windows packages; the Linux tarball uses the distribution's FFmpeg.
Go	1.26+	Only to build from source.
Node.js	24+	Only to build the web app from source.
goose	v3	Migration runner. Bundled in the Docker image; the server binary can also apply migrations itself (`server migrate`).

Operating systems and binaries

- **Docker:** the release image `ghcr.io/collinjaycock/onscreen` is Alpine-based and runs as a non-root `onscreen` user. The NVIDIA image is built locally from `docker/Dockerfile.gpu` (2.3).
- **Release binaries** (GitHub Releases): the server for Linux amd64/arm64, macOS amd64/arm64 and Windows amd64; the standalone worker for Linux amd64/arm64. Binaries are static (CGO disabled).
- **Windows packages:** Windows 10 21H2+, Windows 11 or Windows Server 2019+ (x64).
- **Linux tarball:** x86_64 with glibc 2.17+ or musl.

Sizing

The project does not publish fixed CPU, memory or disk minimums. The defaults scale with the machine, which is a useful guide:

- Concurrent transcodes default to `max(1, CPU cores / 2)` per worker (`TRANSCODE_MAX_SESSIONS`). Software transcoding of 4K is CPU-heavy; a supported GPU (1.4) takes most of that load.
- The scanner runs `CPU cores × 2` file goroutines by default. On a memory-constrained machine, lower it (chapter 11).
- Disk: the cache directory holds resized artwork, seek-bar and chapter thumbnails, subtitle OCR output, converted downloads (capped at 50 GB by default) and optimized versions (no automatic expiry). Put it on a disk with room to grow, and back up the database separately (10.2).
- The NVIDIA image needs driver 570 or newer (CUDA 12.8; Maxwell through Blackwell GPUs).

1.4 Supported hardware encoders

At startup each worker probes FFmpeg and uses only the encoders it can actually run, falling back to software. You can pin the order with `TRANSCODE_ENCODERS` (6.3).

Family	H.264	HEVC	AV1	Platform notes
NVIDIA NVENC	`h264_nvenc`	`hevc_nvenc`	`av1_nvenc`	AV1 on RTX 40-series. Docker needs the NVIDIA Container Toolkit.
Intel Quick Sync (QSV)	`h264_qsv`	`hevc_qsv`	`av1_qsv`	AV1 on Arc and 11th-gen Core and later.
VAAPI	`h264_vaapi`	`hevc_vaapi`	`av1_vaapi`	Linux (Intel and AMD). Needs `libva` and the vendor driver; the run user needs the `video` / `render` groups.
AMD AMF	`h264_amf`	`hevc_amf`	`av1_amf`	Windows. AV1 on RDNA3 discrete GPUs.
Software	`libx264`	`libx265`	`libsvtav1`	Always available.

HDR-to-SDR tone mapping runs on the GPU where possible (6.4).

1.5 Client apps

- **Web**: served by the server itself at its root URL; also the admin interface.
- **Android TV / Google TV and Fire TV**, **Android phone**, **Samsung (Tizen)**, **LG (webOS)**, **Roku**, **Xbox** (a shell over the TV web app the server serves at `/tvapp/`) and the **desktop** app (Windows and Linux).
- `onscreen-cli`: a terminal client that browses a server and plays through mpv.

TV apps sign in by showing a pairing code that a signed-in user claims from the web app or phone. Since v2.6 every app plays the server's play decision (6.1), so the admin settings in this guide apply on all of them.

2 Installation

2.1 Choosing an installation path

Path	Best for	Section
Docker Compose, release image	Most installs: Linux hosts, NAS boxes, anything that runs Docker.	2.2
Docker, NVIDIA GPU image	NVENC/NVDEC transcoding and CUDA HDR tone mapping.	2.3
Windows all-in-one installer	A Windows PC as the server (bundles PostgreSQL, Redis, FFmpeg), or a Windows GPU box as a remote worker.	2.4
Windows portable zip / Linux tarball	Test deployments on real hardware; dependencies through Docker Compose.	2.4, 2.5
From source	Developers and bare-metal installs under systemd.	2.6

> **NOTE**
> The Windows installer, Windows zip and Linux tarball are built from the repository with `make installer-windows-msi`, `make installer-windows` and `make installer-linux`. The GitHub release itself carries the Docker image and plain server/worker binaries.

2.2 Docker Compose (CPU image)

Release images are published as `ghcr.io/collinjaycock/onscreen:<version>`, with `:<major>.<minor>` and `:<major>` tags that follow the latest patch. The examples use `:2.6`; pin a full version (for example `:2.6.0`) if you want upgrades only when you change the tag.

1. Create a project directory and a `.env` file

```
mkdir onscreen && cd onscreen

SECRET_KEY=$(openssl rand -hex 32)
cat > .env <<EOF
DB_PASS=$(openssl rand -hex 24)
SECRET_KEY=${SECRET_KEY}
EOF
```

`DB_PASS` is required (Compose refuses to start without it). `SECRET_KEY` must be a real random key: the server refuses to boot with a placeholder value. Keep a copy of both somewhere safe (3.3).

2. Create `docker-compose.yml`

```
name: onscreen

services:
  postgres:
    image: pgvector/pgvector:pg16   # any PostgreSQL 16+ image works
```

```
    environment:
      POSTGRES_USER: onscreen
      POSTGRES_PASSWORD: ${DB_PASS:?DB_PASS is required}
      POSTGRES_DB: onscreen
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U onscreen"]
      interval: 5s
      timeout: 3s
      retries: 10

  valkey:
    image: valkey/valkey:8-alpine
    ports:
      - "127.0.0.1:6379:6379"
    healthcheck:
      test: ["CMD", "valkey-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10

  # Runs the goose binary and migrations bundled in the image,
  # so the schema always matches the image version.
  migrate:
    image: ghcr.io/collinjaycock/onscreen:2.6
    entrypoint: ["/usr/local/bin/goose"]
    depends_on:
      postgres:
        condition: service_healthy
    command: >
      -dir /migrations postgres
      "postgres://onscreen:${DB_PASS:?DB_PASS is required}@postgres:5432/onscreen?sslmode=disable"
up
    restart: "no"

  server:
    image: ghcr.io/collinjaycock/onscreen:2.6
    depends_on:
      migrate:
        condition: service_completed_successfully
      valkey:
        condition: service_healthy
    environment:
      DATABASE_URL: postgres://onscreen:${DB_PASS:?DB_PASS is required}@postgres:5432/onscreen?
sslmode=disable
      VALKEY_URL: redis://valkey:6379
      SECRET_KEY: ${SECRET_KEY:?SECRET_KEY required}
      # Container loopback is unreachable through a port publish;
      # the host side stays loopback-only.
      METRICS_ADDR: "0.0.0.0:7071"
    restart: unless-stopped
    ports:
      - "7070:7070"
      - "127.0.0.1:7071:7071"
    volumes:
```

```
      - /path/to/your/media:/media:ro
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:7070/health/live || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 10

volumes:
  postgres_data:
```

Replace `/path/to/your/media` with your media folder and point libraries at the container path (`/media/...`). Use `:ro` unless you want OnScreen to write next to your media (NFO files, downloaded artwork, podcast episodes, DVR recordings).

The image stores its cache at `/var/cache/onscreen` (a declared volume). Mount a named volume there if you want resized artwork, thumbnails and converted downloads to survive re-creating the container.

> **NOTE**
>
> **No separate migrate step?** Drop the `migrate` service and its `depends_on` entries, and set `AUTO_MIGRATE: "true"` on the server. It then applies pending migrations at startup, before serving.

3. Start and verify

```
docker compose up -d
curl http://localhost:7070/health/live
docker compose logs -f server
```

The `migrate` service runs once and exits; the server starts after it succeeds. Then open `http://your-server:7070` and complete first-run setup (2.7).

Adding a standalone worker

The server already contains an embedded transcode worker. To add a separate worker container (for example on the same host with a different encoder, or to keep transcoding out of the server process), add a service like this; every worker needs the same `SECRET_KEY` as the server:

```
  worker:
    image: ghcr.io/collinjaycock/onscreen:2.6
    entrypoint: ["/usr/local/bin/worker"]
    depends_on:
      migrate:
        condition: service_completed_successfully
      valkey:
        condition: service_healthy
    environment:
      DATABASE_URL: postgres://onscreen:${DB_PASS:?DB_PASS is required}@postgres:5432/onscreen?
sslmode=disable
      VALKEY_URL: redis://valkey:6379
      SECRET_KEY: ${SECRET_KEY:?SECRET_KEY required}
      # Bind AND advertised address: must be routable from the server.
      WORKER_ADDR: "worker:7073"
    restart: unless-stopped
    healthcheck:   # the image's default probes the server's :7070
```

```
    test: ["CMD", "wget", "--spider", "-q", "http://localhost:7074/health/ready"]
    interval: 10s
    timeout: 3s
    retries: 5
  volumes:
    - /path/to/your/media:/media:ro
```

WARNING

`WORKER_ADDR` is both the address the worker binds and the address the server proxies HLS to. A bare `:7073` means the server's own loopback, so every playlist for a job that worker claims fails with 503. Use a name the server can reach, such as `worker:7073`.

2.3 Docker with an NVIDIA GPU

For NVENC/NVDEC transcoding, build the GPU images. The host needs the NVIDIA Container Toolkit and driver **570 or newer** (`nvidia-smi` should report CUDA 12.8 or higher). The FFmpeg image is deliberately pinned to CUDA 12.8 so it runs on Maxwell through Blackwell GPUs on the drivers most distributions and NAS systems ship.

```
# 1. One-time: FFmpeg with NVENC/NVDEC, scale_cuda, tonemap_cuda and libplacebo
docker build -f docker/Dockerfile.ffmpeg -t onscreen-ffmpeg:latest .

# 2. The application image on top of it
docker build -f docker/Dockerfile.gpu -t onscreen:gpu .
```

Then give the server and/or worker the GPU, for example:

```
worker:
  image: onscreen:gpu
  entrypoint: ["/usr/local/bin/worker"]
  deploy:
    resources:
      reservations:
        devices:
          - driver: nvidia
            count: 1
            capabilities: [gpu, compute, video, graphics, utility]
  environment:
    NVIDIA_VISIBLE_DEVICES: all
    NVIDIA_DRIVER_CAPABILITIES: all
    TRANSCODE_ENCODERS: nvenc,software
  healthcheck:
    test: ["CMD", "wget", "--spider", "-q", "http://localhost:7074/health/ready"]
  volumes:
    - onscreen_cache:/var/cache/onscreen   # must persist
```

`docker/docker-compose.gpu.yml` in the repository is an override that does the same for the server, the worker and a second worker.

> **WARNING**
>
> Two settings silently disable GPU paths if they are missed:
>
> - `NVIDIA_DRIVER_CAPABILITIES` **must include** `graphics` (or be `all`). Without it the Vulkan driver isn't injected and the libplacebo HDR tone map falls back to software. The image sets `all`, but orchestrators (notably TrueNAS apps) can override it; set it explicitly there.
> - `/var/cache/onscreen` **must be persistent.** It holds the CUDA JIT cache. On a fresh volume the first `scale_cuda` use compiles for about 90 seconds; with a persistent volume that happens once.

Verifying GPU acceleration

Each worker logs a `transcode worker ready` line at startup:

```
docker logs <container> 2>&1 | grep "worker ready"
```

Flag	Meaning	Needs
`nvdec_hevc`	NVDEC HEVC decode	NVENC GPU and driver
`cuda_scale`	Decode, scale and encode all in GPU memory (4K SDR)	Matched CUDA/driver and a warm JIT cache
`tonemap_cuda`	All-GPU HDR-to-SDR (4K HDR, no Vulkan)	Matched CUDA/driver and a warm JIT cache
`libplacebo`	Vulkan GPU tone map and scale	The `graphics` capability

`cuda_scale` and `tonemap_cuda` are probed in the background: they read `false` in the first line and switch on later (about 90 seconds on a cold cache, about 2 seconds on a warm one), logged as `cuda_scale enabled` and `tonemap_cuda enabled`. `libplacebo` can stay `false` where Vulkan won't start (TrueNAS, WSL2); HDR then uses `tonemap_cuda`.

2.4 Windows installer and portable zip

All-in-one installer (`OnScreen-Setup-<version>.exe`)

Built with `make installer-windows-msi` (Inno Setup). It bundles the server and worker, FFmpeg (Gyan.dev full build with NVENC, QSV, AMF and AV1), WinSW, portable PostgreSQL 17 and Redis for Windows. The wizard offers two modes:

- **Full** (default): installs to `C:\Program Files\OnScreen`, initialises a PostgreSQL cluster in `%ProgramData%\OnScreen\pgdata` (with `scram-sha-256` authentication), generates a random database password and `SECRET_KEY`, writes `.env`, applies migrations, and registers three Windows services (PostgreSQL, Redis, OnScreen). It then opens `http://localhost:7070` for setup.
- **Worker**: joins an existing server's fleet. Enter the primary's `DATABASE_URL`, `VALKEY_URL` and `SECRET_KEY` and this machine's `WORKER_ADDR`. The worker runs as an at-logon *interactive scheduled task* named `OnScreenWorker`, not a service, because a service runs in session 0 without GPU access.

The install folder and `%ProgramData%\OnScreen` are restricted to SYSTEM and Administrators (Users read/execute on the program folder); `.env` and the service files are SYSTEM and Administrators only.

Uninstalling stops and removes the services and task, and asks whether to delete the database and logs.

Portable zip (`onscreen-windows-amd64-<version>.zip`)

Built with `make installer-windows`. It runs PostgreSQL and Valkey in Docker Desktop:

```
# elevated PowerShell
cd "C:\Program Files\OnScreen"
copy .env.example .env      # set SECRET_KEY at minimum
.\start-deps.ps1            # Postgres + Valkey on 127.0.0.1 via Docker Desktop
.\start.ps1                 # applies migrations, runs in the foreground
.\install-service.ps1       # optional: register the OnScreen Windows service
```

Extract under `C:\Program Files`: a folder directly under `C:\` inherits write access for every signed-in user, which would let any local account replace the binaries of a service running as LocalSystem. `install-service.ps1` resets the folder permissions anyway. Logs go to `logs\onscreen.out.log` and `logs\onscreen.err.log`; manage the service with `Restart-Service OnScreen`.

2.5 Linux portable tarball

Built with `make installer-linux`. It contains `server`, `worker`, `goose`, the migrations, `migrate.sh`, start and service scripts, and a Compose file for PostgreSQL and Valkey. It does **not** bundle FFmpeg: install your distribution's (`sudo apt install ffmpeg`), which already uses the right VAAPI driver for your GPU.

```
tar -xzf onscreen-linux-amd64-<VERSION>.tar.gz
cd onscreen-linux-amd64-<VERSION>
chmod +x *.sh server worker goose
cp .env.example .env && ${EDITOR:-nano} .env   # SECRET_KEY at minimum
./start-deps.sh                                 # Postgres + Valkey in Docker
./start.sh                                      # migrates, then runs in the foreground
sudo ./install-service.sh                       # optional: systemd unit "onscreen"
```

The systemd unit runs as the invoking (sudo) user, applies migrations in `ExecStartPre`, and is hardened (`NoNewPrivileges`, `ProtectSystem=full`, `ProtectHome=read-only` and more). `install-service.sh` makes `.env` mode 0600. To update, stop the service, replace `server` and `worker`, and start it again; migrations run on start.

For VAAPI, add the run user to the `video` (and on some distributions `render`) group, and check that `vainfo` lists `VAEntrypointEncSlice`.

2.6 Building from source (bare metal)

```
# Debian/Ubuntu dependencies
sudo apt install postgresql-16 valkey ffmpeg

git clone https://github.com/CollinJAycock/OnScreen.git
cd OnScreen
make build      # web UI + TV app, copied into the embed dirs, then bin/server and bin/worker
```

Use `make build` rather than a bare `go build`: the server embeds the web UI from `internal/webui/dist`, and `make build` refreshes that copy.

Create the database and run migrations:

```
CREATE USER onscreen WITH PASSWORD 'your-password';
CREATE DATABASE onscreen OWNER onscreen;

make migrate DATABASE_URL="postgres://onscreen:pass@localhost:5432/onscreen?sslmode=disable"
# or: DATABASE_URL=... ./bin/server migrate
```

Put the configuration in an environment file and run the server (and, optionally, a worker) under systemd:

```
# /etc/systemd/system/onscreen-server.service
[Unit]
Description=OnScreen Server
After=network.target postgresql.service valkey.service

[Service]
Type=simple
User=onscreen
Group=onscreen
EnvironmentFile=/etc/onscreen/env
ExecStart=/usr/local/bin/server
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

A matching `onscreen-worker.service` runs `/usr/local/bin/worker`. Enable both with `sudo systemctl enable --now onscreen-server onscreen-worker`.

> **NOTE**
> The server also reads a `.env` file on its own: first in the working directory, then next to the executable. Variables already set in the environment (by systemd, Docker, WinSW or the shell) always win over the file.

2.7 First-run setup

On a fresh install the users table is empty, and whoever completes setup becomes the admin. Open the server in a browser and create the admin account.

To keep a stranger from claiming a freshly exposed server, first-run setup is accepted only when:

- the request comes from the local network: loopback, private ranges, link-local, Tailscale/CGNAT (`100.64.0.0/10`) or this host's own IPv6 /64s; **and**
- the browser addressed the server by IP, `localhost`, a single-label name, or a local suffix (`.local`, `.lan`, `.home`, `.home.arpa`, `.internal`).

Set `ALLOW_PUBLIC_SETUP=true` only if you must finish setup over the internet, and finish it promptly.

> **WARNING**
>
> Behind Docker Desktop, rootless Podman or docker-proxy on the IPv6 path, every client can appear to come from the private bridge gateway, so an internet client could pass the local check. Finish setup immediately after the first start and don't publish an unconfigured instance.

After setup, the usual next steps are: set the **Public URL** (Settings ▸ General ▸ Server), add metadata keys (5.5), create libraries (5.2), check the encoders on the Transcode page (6.2), turn on two-factor sign-in for your account (7.5), and set up remote access (chapter 4).

3 Configuration reference

3.1 How configuration works

OnScreen is configured in two places:

- **Environment variables** hold the bootstrap set the process needs before it can read the database: connection strings, `SECRET_KEY`, `NODE_ID`, bind addresses, paths and a few switches.
- **The admin Settings pages** hold everything else, stored in the database: the public URL, log level, CORS list, metadata and service keys, SSO, SMTP, transcode tuning, scheduled tasks, the worker fleet, tracing and more.

Many environment variables are only the *initial default* for a value you can also edit in the UI; a saved UI value wins.

- **Per node** (Settings ▸ General ▸ Nodes): `LISTEN_ADDR`, `METRICS_ADDR`, `CACHE_PATH`, `WORKER_HEALTH_ADDR`, `STATIC_ABR_ROOT`, `SITE_ID`, `TRANSCODE_QSV_DECODE`, `DISABLE_EMBEDDED_WORKER`.
- **Cluster-wide** (Settings ▸ General ▸ System): `RETAIN_MONTHS`, `SERVER_NAME`, `DISCOVERY_ENABLED`, `DISCOVERY_PORT`, `TMDB_RATE_LIMIT`, `PUBLIC_ASSET_CACHE`, `STATIC_ABR_ENABLED`, and the scan settings.
- **Settings ▸ Transcode**: the NVENC tuning, output ceilings and adaptive-bitrate ladder.
- **Environment only**: `NODE_ID`, `IGNORE_NODE_DB_CONFIG`, the connection strings and every other variable not listed above.

> **WARNING**
>
> **Nothing is hot-reloadable.** Changes to environment variables and to restart-required UI settings take effect when the process restarts. `SIGHUP` is handled but only logs a warning saying no setting was reloaded.

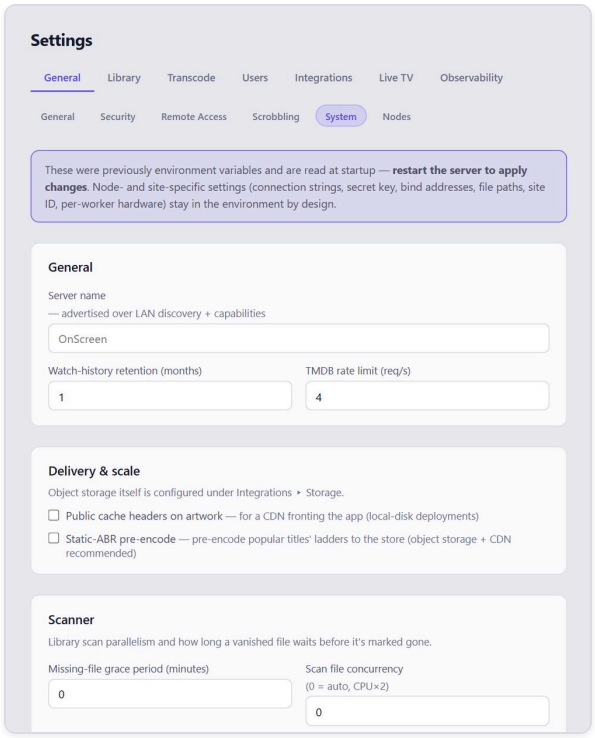


Figure 3-1. Settings ▸ General ▸ System: cluster-wide values shared by every node.

If a bad value saved under Nodes (an unreachable listen address, for example) locks a node out, start it with `IGNORE_NODE_DB_CONFIG=true` to boot from environment and defaults only.

3.2 Environment variables

Defaults are the values in `internal/config/config.go`. An empty, zero or unparseable rate-limit value falls back to its default.

Required

Variable	Default	Meaning
`DATABASE_URL`	(required)	PostgreSQL connection string. Accepts a multi-host failover DSN (`...@primary:5432,standby:5432/onscreen?target_session_attrs=read-write`).
`VALKEY_URL`	(required)	Valkey/Redis connection string, e.g. `redis://localhost:6379` .
`SECRET_KEY`	(required)	32-byte key for token signing and AES-256-GCM encryption of stored secrets. Hex (64 chars), base64 (~44 chars) or a raw string of 32+ characters. See 3.3.

Database, cache and node identity

Variable	Default	Meaning
`DATABASE_RO_URL`	`DATABASE_URL`	Read-replica connection string for query offloading.
`CACHE_PATH`	`~/.onscreen/cache/artwork` (image: `/var/cache/onscreen`)	Root of every on-disk cache: artwork, thumbnails, subtitles and OCR, Live TV, converted downloads (`downloads/`), optimized versions (`optimized/`).
`NODE_ID`	host name	Stable identity that keys this node's row under Settings ▸ Nodes.
`IGNORE_NODE_DB_CONFIG`	`false`	Break-glass: ignore this node's saved Nodes settings.
`AUTO_MIGRATE`	`false`	Apply pending migrations at startup, before serving. For installs without a separate migrate step.

Server and network

Variable	Default	Meaning
`LISTEN_ADDR`	`:7070`	HTTP(S) API and web UI bind address.
`METRICS_ADDR`	`127.0.0.1:7071`	Prometheus `/metrics` and unauthenticated `/debug/pprof`. Keep private; in a container set `0.0.0.0:7071` and publish it to host loopback only.
`TLS_CERT_FILE` / `TLS_KEY_FILE`	(none)	PEM certificate chain and key for built-in HTTPS. Both or neither; setting one is a startup error.
`TRUSTED_PROXIES`	(none)	Comma-separated IPs/CIDRs allowed to set `X-Forwarded-*`. Unset trusts any loopback or private-network peer. See 4.3.
`PUBLIC_SEGMENT_BASE_URL`	(none)	Host that HLS playlists and segments are fetched from instead of the main host (4.5).
`ALLOW_PUBLIC_SETUP`	`false`	Allow first-run admin creation from a public address.
`SERVER_NAME`	`OnScreen`	Name shown in LAN discovery and capability responses.
`DISCOVERY_ENABLED`	`true`	Answer LAN auto-discovery broadcasts.
`DISCOVERY_PORT`	`7368`	UDP port for LAN discovery.
`RETAIN_MONTHS`	`24`	Months of watch history to keep.

RTMP ingest ("go live")

Variable	Default	Meaning
`RTMP_ENABLED`	`true`	Embedded RTMP server that turns OBS/ffmpeg pushes into Live TV channels. A bind failure is logged and the rest of the server still starts.
`RTMP_LISTEN_ADDR`	`:1935`	RTMP listen address; must be reachable by broadcasters.
`RTMP_PUBLIC_HOST`	(none)	Host name shown in the ingest URL in the admin UI; empty uses the request's host.

Transcoding and workers

Variable	Default	Meaning
`TRANSCODE_MAX_SESSIONS`	`max(1, cores/2)`	Concurrent transcode sessions on this worker. Per-worker caps can also be set in the fleet section of Settings ▸ Transcode.
`TRANSCODE_ENCODERS`	auto-detect	Encoder priority, e.g. `nvenc,software` (6.3).
`DISABLE_EMBEDDED_WORKER`	`false`	Server only: don't run the in-process worker; use standalone workers.
`WORKER_ADDR`	`:7073`	Standalone worker: segment-server bind address *and* the address it registers for the server to proxy to. Must be routable from the server.
`WORKER_HEALTH_ADDR`	`:7074`	Standalone worker health server (`/health/live` , `/health/ready`).
`TRANSCODE_MAX_BITRATE_KBPS`	`40000`	Output bitrate ceiling (initial default for the UI value).
`TRANSCODE_MAX_WIDTH` / `_HEIGHT`	`3840` / `2160`	Output resolution ceiling (initial default).
`TRANSCODE_ABR`	`false`	Serve an adaptive-bitrate ladder instead of a single rendition.
`TRANSCODE_ABR_MAX_HEIGHT`	`0`	Hard cap on the ladder's top rung; 0 = source resolution.
`TRANSCODE_ABR_AUTO_MAX_HEIGHT`	`1080`	Top rung when the viewer hasn't picked a quality; 0 = source.
`TRANSCODE_NVENC_PRESET`	`p4`	`p1` (fastest) to `p7` (best quality).
`TRANSCODE_NVENC_TUNE`	`hq`	`hq` , `ll` or `ull` .
`TRANSCODE_NVENC_RC`	`vbr`	`vbr` , `cbr` or `constqp` .
`TRANSCODE_MAXRATE_RATIO`	`1.5`	Peak-to-average bitrate ratio for all encoders.
`TRANSCODE_QSV_DECODE`	`false`	Intel Quick Sync HEVC decode into system memory. Opt in once the worker's QSV stack is known good.

Variable	Default	Meaning
`TRANSCODE_QSV_VRAM`	`true`	Keep decode, scale and encode in GPU memory when a QSV encoder is used (SDR). Falls back to software decode per job.
`TRANSCODE_VAAPI_VRAM`	`true`	The VAAPI equivalent of `TRANSCODE_QSV_VRAM`.
`TRANSCODE_ENCODER_FAILOVER`	`true`	When a hardware encoder can't get the GPU (e.g. GeForce NVENC session cap), retry on the next encoder the box has (QSV, then software).

Converted downloads (new in v2.6)

Variable	Default	Meaning
`DOWNLOAD_MAX_CONVERSIONS`	`1`	Conversions (downloads and optimized versions) encoding at once on this server: 1 or 2.
`DOWNLOAD_CACHE_MAX_GB`	`50`	Size limit of `CACHE_PATH/downloads`; the hourly cleanup evicts least-recently-used files past it.
`DOWNLOAD_CACHE_TTL_HOURS`	`168`	How long a finished file is kept after it was last requested or fetched.

DVR (new in v2.6)

Variable	Default	Meaning
`ONSCREEN_DVR_POSTPROCESS_SCRIPT`	(none)	Program run after each recording is post-processed, with the recording's path as its only argument (no shell).
`ONSCREEN_COMSKIP_PATH`	(none)	Path to `comskip` for commercial detection. Empty uses the built-in detector.
`ONSCREEN_COMSKIP_INI`	(none)	Passed to comskip as `--ini`.

These executables can only be set in the environment, never in the UI: a UI-settable program path would be code execution for anyone holding an admin session.

Scanning and metadata

Variable	Default	Meaning
`SCAN_FILE_CONCURRENCY`	`cores × 2`	Concurrent file-scan goroutines (initial default; Settings ▸ System).
`SCAN_LIBRARY_CONCURRENCY`	`2`	Libraries scanned at once.
`MISSING_FILE_GRACE_PERIOD`	`15m`	Wait before a missing file is marked unavailable.
`TMDB_API_KEY`	(none)	TMDB v3 key; seeded into Settings on first run.
`TMDB_RATE_LIMIT`	`5`	TMDB requests per second.
`TVDB_API_KEY`	(none)	TheTVDB v4 key; seeded into Settings on first run.

Rate limits

Variable	Default	Meaning
`OS_AUTH_RATE_LIMIT_PER_MIN`	`10`	Requests per minute per address to the credential endpoints (sign-in, PIN switch, pairing claims).
`OS_AUTH_RATE_LIMIT_FAIL_CLOSED`	`false`	When Valkey is unreachable, refuse those requests with 503 instead of letting them through unlimited.
`OS_TRANSCODE_START_RATE_LIMIT_PER_MIN`	`10`	Transcode session starts per minute.

High availability and scale (all optional)

Variable	Default	Meaning
`VALKEY_SENTINEL_ADDRS`	(none)	Comma-separated Sentinel addresses. When set, Valkey is reached through Sentinel; `VALKEY_URL` still supplies auth, db and TLS.
`VALKEY_SENTINEL_MASTER`	`onscreen`	Sentinel master name.
`VALKEY_SENTINEL_PASSWORD`	(none)	Password for the Sentinels themselves.
`PUBLIC_ASSET_CACHE`	`false`	Send resized artwork as `Cache-Control: public` for a CDN. Only with a CDN in front.
`STATIC_ABR_ENABLED`	`false`	Pre-encode the ABR ladder of the most-played titles to the media store.
`STATIC_ABR_ROOT`	(none)	Key prefix (object storage) or directory for static-ABR output.
`SITE_ID`	(none)	This deployment's site name, reported on `/health/cluster`.

There are no environment variables for the public URL, log level, CORS list, SSO, SMTP, object storage or tracing; they live in the Settings pages. `DEV_FRONTEND_URL` exists for development builds only and is ignored in production.

3.3 Secrets and key rotation

`SECRET_KEY`

The key signs access, stream and segment tokens and encrypts every secret OnScreen stores: metadata and service API keys, SSO and SMTP configuration, object-storage credentials, an uploaded TLS key, webhook signing secrets, notification-agent credentials, Radarr/Sonarr keys, users' TOTP secrets and scrobbling tokens.

- Generate one with `openssl rand -hex 32`. The server refuses keys shorter than 32 bytes, keys with too little entropy, and keys that look like placeholders (containing `change-me`, `example`, `your-secret`, `placeholder`, `dev-secret` and similar).

- Every server and worker in a fleet must use the **same** key: workers' segment servers authenticate the server with a bearer derived from it.
- **Back it up out of band.** Without it, encrypted settings and existing tokens cannot be recovered from a database backup.

Rotating `SECRET_KEY`

You can't just change the key and restart: the new key can't decrypt the old secrets. Use the `rotate-key` tool (built from `cmd/rotate-key`), which re-encrypts everything in one transaction:

```
# 1. New key
openssl rand -hex 32

# 2. Dry run: reports a per-table summary, commits nothing
OLD_SECRET_KEY=<current> SECRET_KEY=<new> DATABASE_URL=<dsn> go run ./cmd/rotate-key

# 3. Apply (rolls back on any error)
OLD_SECRET_KEY=<current> SECRET_KEY=<new> DATABASE_URL=<dsn> go run ./cmd/rotate-key -apply

# 4. Restart every server and worker with the new SECRET_KEY
```

If the dry run shows `rotated=0` for everything, `OLD_SECRET_KEY` isn't the key in use; stop and fix it. Do steps 3 and 4 in a maintenance window with the server stopped. Afterwards every user signs in again, in-flight streams reconnect, and links in digest emails already sent stop working. Values the old key can't decrypt are left untouched and reported as skipped.

Rotating to the *same* key is a safe way to upgrade every stored secret to the current encryption envelope (`encv2:`).

`DB_PASS`

The Compose files require `DB_PASS` and have no default. An existing PostgreSQL volume keeps the password it was created with: set `DB_PASS` to that value, or change it first with `ALTER ROLE onscreen PASSWORD '...'`.

3.4 Database and migrations

Schema migrations are managed with goose and embedded in the server binary. The server **does not** migrate on its own unless `AUTO_MIGRATE=true`; until pending migrations are applied, `GET /health/ready` stays unready (and the Docker health check, which probes it, reports unhealthy).

Ways to apply migrations:

```
# the Compose "migrate" service (2.2), or:
make migrate DATABASE_URL="postgres://onscreen:pass@localhost:5432/onscreen?sslmode=disable"
goose -dir internal/db/migrations postgres "$DATABASE_URL" up
docker run --rm --network host --entrypoint /usr/local/bin/goose \
  ghcr.io/collinjaycock/onscreen:2.6 -dir /migrations postgres "$DATABASE_URL" up
DATABASE_URL=... ./bin/server migrate          # the binary embeds the migrations
docker exec <container> /usr/local/bin/server migrate

# status
goose -dir internal/db/migrations postgres "$DATABASE_URL" status
```

The Windows installer, the Windows zip's `start.ps1` / `install-service.ps1` and the Linux tarball's `migrate.sh` (run by `start.sh` and the systemd unit) apply migrations for you.

If the first migration fails creating an extension, create `pg_trgm`, `pgcrypto` and `unaccent` once as a superuser (on distributions that split them out, install the PostgreSQL contrib package).

4 Networking and remote access

4.1 Ports

Port	Variable	Purpose	Exposure
`7070/tcp`	`LISTEN_ADDR`	API and web UI	Public, behind TLS.
`127.0.0.1:7071`	`METRICS_ADDR`	Prometheus metrics and **unauthenticated** pprof	Private. pprof can leak live tokens.
`1935/tcp`	`RTMP_LISTEN_ADDR`	RTMP ingest	Reachable by broadcasters; publishes need a stream key.
`7073/tcp`	`WORKER_ADDR`	Standalone worker segment server	Reachable from the server; needs the `SECRET_KEY` bearer.
`7074/tcp`	`WORKER_HEALTH_ADDR`	Worker health checks	Private.
`7368/udp`	`DISCOVERY_PORT`	LAN auto-discovery	LAN only; disable on untrusted segments.

4.2 Reverse proxy

In production, put OnScreen behind a reverse proxy that terminates TLS (or use built-in HTTPS, 4.4). OnScreen uses no WebSockets; its one long-lived connection is the server-sent events stream at `/api/v1/notifications/stream`, which needs buffering off and a long read timeout.

nginx

```
upstream onscreen { server 127.0.0.1:7070; }

server {
    listen 80;
    server_name media.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name media.example.com;
    ssl_certificate     /etc/letsencrypt/live/media.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/media.example.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    client_max_body_size 0;
    proxy_buffering off;
    proxy_request_buffering off;

    location / {
        proxy_pass http://onscreen;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
```

```
        # nginx is the edge: REPLACE any client-sent X-Forwarded-For.
        proxy_set_header X-Forwarded-For $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_http_version 1.1;
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
    }

    location /metrics { deny all; }
}
```

For a fuller configuration start from `docker/nginx.conf` in the repository. It keeps every `proxy_set_header` at server level, because nginx drops all inherited `proxy_set_header` lines in a location that sets one of its own. Losing `X-Forwarded-For` that way makes every client look like nginx's private address, which opens the local-only setup gate to the internet and collapses all per-IP rate limits into one bucket. Don't add `proxy_set_header` or `add_header` inside a location. Behind a CDN in front of nginx, use `real_ip_header` / `set_real_ip_from`.

Caddy

```
media.example.com {
    reverse_proxy localhost:7070 {
        flush_interval -1
    }
}
```

Whatever the proxy, set `TRUSTED_PROXIES` to its address (next section) and set the **Public URL** in Settings ▸ General ▸ Server, then restart.

4.3 `TRUSTED_PROXIES` and client addresses

OnScreen uses the client's address for rate limits, the audit log, the first-run setup gate and, since v2.6, for deciding whether a viewer is at home or on the internet (4.7). It reads `X-Forwarded-For` / `X-Real-IP` only from peers it trusts:

- **Unset** (default): any loopback or private-network peer is trusted. This keeps the common "proxy in a sibling container" setup working, but lets any LAN client forge its address (dodging login rate limits, forging the audit-log address). Private addresses in `X-Forwarded-For` are also treated as proxy hops, so LAN clients behind a proxy that sends only `X-Forwarded-For` share one rate-limit bucket.
- **Set** to your proxy's address(es), e.g. `TRUSTED_PROXIES=127.0.0.1,172.18.0.0/16`: only those peers may set forwarding headers.

> **WARNING**
> Behind a reverse proxy or Cloudflare, the internet streaming limit works only if the proxy forwards the viewer's real address (the bundled nginx sample and `cloudflared` do) **and** the proxy's address is in `TRUSTED_PROXIES`. Otherwise every viewer looks like the proxy, which is usually on your network, and the limit never applies.

4.4 HTTPS

Three options:

1. **A reverse proxy** (4.2). The best choice if you want automatic renewal (Caddy is the simplest).
2. **Certificate files:** set `TLS_CERT_FILE` and `TLS_KEY_FILE` (PEM). The server then serves HTTPS on `LISTEN_ADDR` (often changed to `:443`). These win over an uploaded certificate.
3. **Upload in the UI:** paste the certificate and key under Settings ▸ General ▸ Security ▸ *HTTPS / TLS certificate*. They are stored encrypted and loaded at startup (restart required). This applies to the whole cluster.

The server never renews certificates itself; with options 2 and 3, replace the certificate and restart on renewal (for example after `certbot certonly`).

4.5 Cloudflare Tunnel and split segment access

Some remote-access setups can't carry bulk video through the main host name. A Cloudflare Tunnel, for example, has terms that restrict proxying self-hosted video and can throttle long streams. Set `PUBLIC_SEGMENT_BASE_URL` to a host that reaches **the same server** by a direct path (a DNS-only record, a port-forwarded `stream.` subdomain, a Tailscale name):

```
PUBLIC_SEGMENT_BASE_URL=https://stream.example.com
```

HLS manifests still come from the main host, but every segment and rung-playlist URL inside them points at `https://stream.example.com/api/v1/transcode/...`. The direct host must terminate TLS and reach the same OnScreen origin. Direct play (`/media/stream/...`), artwork and subtitles stay on the main host.

With a tunnel, remember `TRUSTED_PROXIES` (4.3): `cloudflared` forwards the viewer's address, but OnScreen only believes it from a trusted peer.

4.6 Automatic router port mapping

> **NEW IN v2.6**
>
> Settings ▸ General ▸ Remote Access can ask your router to forward a public port to the server, using UPnP (IGD v1 and v2), PCP or NAT-PMP.

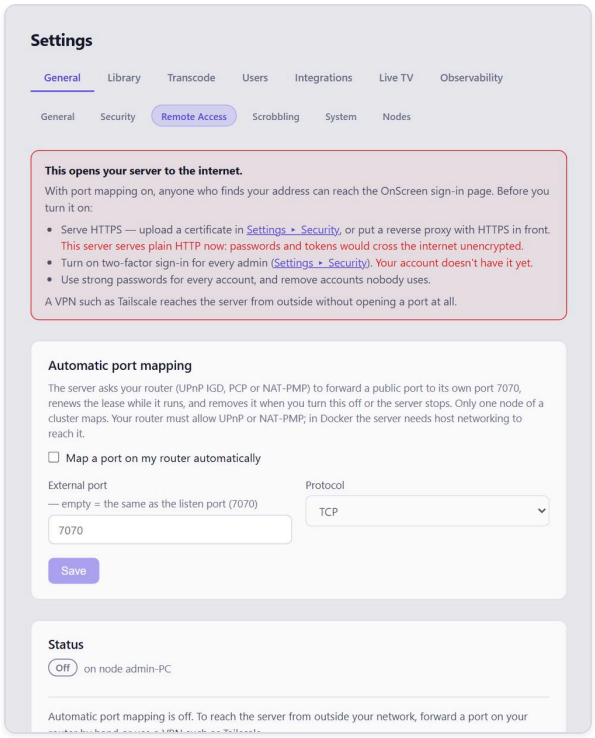


Figure 4-1. Settings ▸ General ▸ Remote Access, with its warning and the mapping status.

- **Off by default.** Turning it on exposes OnScreen to the whole internet. Serve HTTPS and turn on two-factor sign-in for every admin first; the page warns when either is missing. A VPN such as Tailscale reaches the server without opening a port at all.
- Only the API listen port is mapped (you choose only the public port), TCP only, and never when the server listens on loopback only. In a cluster only one node maps.
- The lease is one hour, renewed at half its lifetime, and removed when you turn the option off, change the port, or the server stops.
- In Docker the server needs host networking to reach the router.
- The server never tests its own reachability through an outside service. The Status card shows the router's public address and port; test from outside your network. A private "public" address means carrier-grade NAT or a second router, where mapping can't help.

4.7 The home network and the internet limit

A viewer counts as **at home** when their address is loopback, private (`192.168.x` , `10.x` , `172.16–31.x` , `fd00::/8`), link-local, inside this server's own IPv6 network, or in `100.64.0.0/10` (so Tailscale devices count as home). Add more ranges, such as a VPN or office subnet, under Settings ▸ Transcode ▸ *Additional local networks*.

Everyone else is **remote**, and can be held to a lower bitrate with the internet streaming limit (6.6). Now Playing labels each stream LAN or remote.

4.8 LAN discovery and CORS

The server answers LAN discovery on UDP 7368 so apps can find it. It replies only to loopback, private and link-local senders, with the server name, machine ID and version, unauthenticated. Turn it off with `DISCOVERY_ENABLED=false` (or in Settings ▸ System) on untrusted networks.

CORS allowed origins (Settings ▸ General ▸ Server) controls cross-origin browser access to the API. The TV and desktop apps' origins are always allowed and don't need to be listed. The list is read at startup.

5 Libraries

5.1 Library types

Type	Holds	Metadata
Movies	Films, with editions and multi-part rips	TMDB
TV Shows	Show → season → episode	TMDB, TheTVDB fallback
Anime	Shows with anime naming rules	AniList first, then TMDB/TheTVDB
Cartoons	Shows, kept as their own library type	TMDB
Music	Artist → album → track	File tags, MusicBrainz, Cover Art Archive
Audiobooks	Author → series → book → chapter	File tags and folders
Podcasts	RSS subscriptions or local files	The feed
Photos	Photos and camera RAW, with EXIF	EXIF
Home Videos	Personal videos, one file per item	File name and date
Books	EPUB, CBZ, CBR (and PDF in the OPDS catalog)	File
DVR Recordings	Live TV recordings	The program guide

A manga library type exists in the code but is hidden: the API and the New Library page don't offer it.

5.2 Creating and configuring a library

Choose **New Library** on the home page. Give it a name, a type, one or more **scan paths** (as the server sees them; in Docker, the container path), the metadata **agent** (TMDB or None; DVR and podcast libraries default to None) and the **language**.

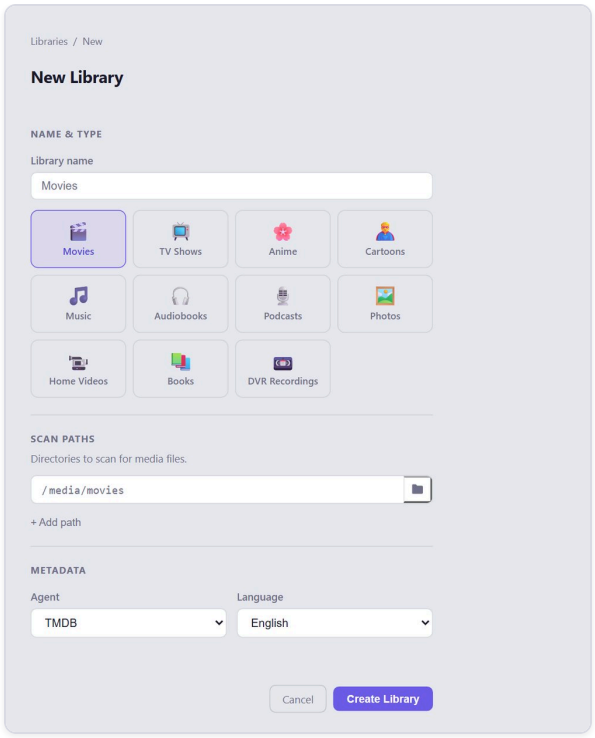


Figure 5-1. Creating a library.

A library's **Settings** page (from the library's header) has further options, depending on the type: scan interval, seek-bar thumbnails, automatic subtitle download and its languages, episode order, collections from NFO files, saving NFO files, camera RAW pairing, face data, intro and credits detection, and visibility (a private library is visible only to users granted access).

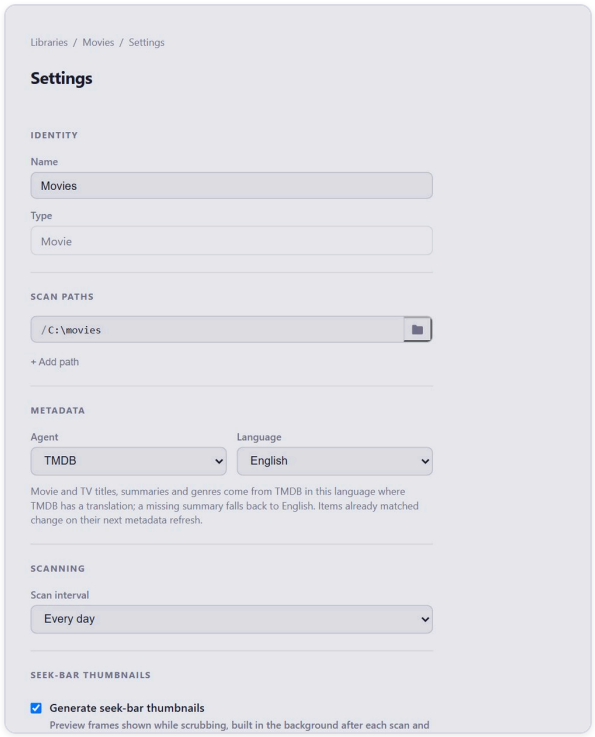


Figure 5-2. A movie library's settings: identity, scan paths, metadata agent and language, scan interval, seek-bar thumbnails.

5.3 Folder and file naming

OnScreen reads the conventions Plex, Jellyfin, Kodi and Sonarr/Radarr use. Keep one title per folder where you can.

Movies

```
Movies/
  Blade Runner (1982)/
    Blade Runner (1982) {edition-Final Cut}.mkv
    Blade Runner (1982) - Theatrical.mkv
  Some Movie (2001)/
    Some Movie (2001) cd1.mkv
    Some Movie (2001) cd2.mkv
```

- **Editions** (labels in the version picker): Plex's `{edition-...}` in the file or folder name, Jellyfin's `Title (Year) - Label` suffix, and common tags after the year such as `[Extended]`, Director's Cut, Extended Edition, Theatrical, Unrated, Uncut, IMAX, Remastered.
- **Multi-part movies**: `cd`, `part`, `pt`, `disc`, `disk` or `dvd` with a number, or `-a` / `-b`, after the year. They play as one movie on every app (the server joins the parts, so they are remuxed or transcoded, never direct-played). Part words *before* the year ("Dune Part One (2021)") are part of the title.

TV shows, anime and cartoons

```
TV Shows/
  Show Name (2005)/
    Season 01/
      Show Name - S01E01 - Pilot.mkv
      Show Name - S01E02-E03.mkv        (one multi-episode file)
    Specials/
      Show Name - S00E01.mkv
```

- Episodes: `S01E03` or `1x03`. Season folders: `Season 1`, `Season 01` or `S01`; `Season 00` and a `Specials` folder are season 0.
- **Multi-episode files** (new in v2.6): `S01E01E02`, `S01E01-E02`, `S01E01-02`, `S01E01.S01E02`, `1x01-1x02`, `1x01-02`. Shown as "E1–E2" and counted as both episodes.
- **Anime** absolute numbering: `[Group] Title - 01 [1080p].mkv` (spaced dash). The season comes from the `Season N` folder, else season 1.
- Include the year (or a Sonarr qualifier like `(US)`) in the show folder: since v2.6 a show must match its folder's name and year before it is accepted automatically.

Music

Tags come first (ID3, Vorbis comments, MP4; MusicBrainz IDs and ReplayGain included). Without tags the folder layout `Artist/Album/01 - Track.ext` is used. Multi-disc albums are recognised from `DISCNUMBER` tags, `CD1` / `Disc 2` folders, or `1-05 Title` file names. Featured artists come from Picard's `ARTISTS` tag or "feat." in the artist or title.

Audiobooks

```
Audiobooks/Author/Series/Book/01 - Chapter.mp3   author → series → book → chapters
Audiobooks/Author/Book/...                       author → book
Audiobooks/Book.m4b                              a single-file book
```

A `cover.jpg` or `folder.jpg` in the book's folder is used as its cover.

Books, home videos, photos, podcasts

- **Books**: one file (EPUB, CBZ, CBR) is one book; the title comes from the file name.
- **Home videos**: one file is one item; a leading date in the file name becomes its date, and files in a subfolder are grouped into a collection named after it.
- **Photos**: JPEG, HEIC, TIFF and camera RAW (CR2, CR3, NEF, ARW, DNG, RAF, ORF, RW2, PEF). A RAW and a JPEG with the same name in the same folder become one photo unless you turn off *Pair RAW files with their JPEGs*.
- **Podcasts** subscribed by RSS download into `<podcast>/<date> <episode title>.mp3` inside the library folder; the *Refresh podcast feeds* task checks feeds hourly. Local podcast files use folder = show, file = episode.

5.4 Scanning

- Scan a library with **Scan** on its page, or let its *Scan interval* run it. A `scan_library` scheduled task can also scan all or one library on a cron schedule.
- A file that disappears is marked unavailable after `MISSING_FILE_GRACE_PERIOD` (15 minutes).
- Files that are really HLS/DASH/concat/SDP playlists are refused at scan and playback (`UNSUPPORTED_CONTAINER`).
- The banner at the top of the web app (“Filling in artwork”) reports items still waiting for artwork; the *Refresh missing artwork* task retries every two hours.

5.5 Metadata agents and API keys

Service keys are stored encrypted in the database and edited in Settings ▸ General (they are not environment variables, except that `TMDB_API_KEY` / `TVDB_API_KEY` seed the first value).

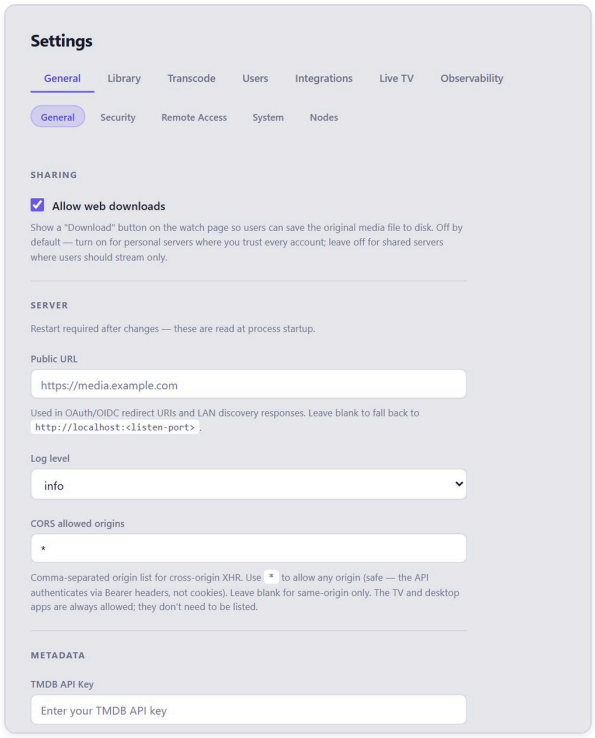


Figure 5-3. Settings ▸ General: sharing, server (public URL, log level, CORS) and the start of the Metadata keys.

Key (Settings ▸ General)	Used for	Without it
TMDB API Key	Movie and TV matching, artwork, ratings, genres, studios, franchise collections, Discover/requests	No movie/TV metadata.
TheTVDB API Key	Episode metadata fallback; DVD and absolute episode orders	TMDB only.
OMDb API Key + OMDb Daily Lookups (v2.6)	IMDb, Rotten Tomatoes and Metacritic scores; the hourly task stays within the daily limit (1,000 by default)	No external scores.
fanart.tv API Key (v2.6)	Clear logos, banners, disc art	Local files and AniList banners only.
OpenSubtitles (API key, optional username/password)	Subtitle search in the player and automatic subtitle download	No online subtitles.
Arr Notifications API Key	Authenticates Radarr/Sonarr webhooks to OnScreen	No arr webhooks.
Scrobbling apps (Last.fm, Trakt)	App credentials users' scrobbling links run under	No Last.fm/Trakt.

AniList (anime) and MusicBrainz (music) need no key. Logos, banners and disc art are taken first from files next to your media with Kodi's names (`clearlogo.png` / `logo.png` , `banner.jpg` , `disc.png` / `discart.png` / `cdart.png`).

5.6 Language and episode order

Metadata language (library settings): movie and TV titles, summaries, genres, seasons and episodes come from TMDB in this language where TMDB has a translation; a missing summary falls back to English. Searches stay in English, so automatic matching is unaffected.

> **NEW IN v2.6**
> The library language was stored but ignored before v2.6. Items already matched change on their next metadata refresh.

Episode order (TV, anime and cartoon libraries): *Aired* (default), *DVD* or *Absolute*, with a per-show override on the show's page. It uses TMDB episode groups for shows matched on TMDB and TheTVDB's orders for shows matched there; a show whose provider has no such order keeps aired order. Changing it refreshes the affected shows' episodes in the background.

5.7 NFO files

- **Reading:** an NFO's `<studio>` tags win over TMDB's. A movie library can import Kodi `<set>` (and optionally `<tag>`) groupings as collections: *Collections from NFO files*, off by default; a daily task picks up edited files. Sets that are TMDB film series OnScreen already groups are skipped.
- **Writing** (v2.6): *Save metadata as NFO files next to media* (movie and TV libraries, off by default) writes Kodi-compatible `.nfo`, `tvshow.nfo` and episode NFO files after a title is enriched, edited or re-matched. Only files OnScreen wrote (marked `<!-- written by OnScreen -->`) are ever replaced; a read-only folder only logs a warning. Not available with object storage.

5.8 Editing metadata, Fix Match and labels

Metadata editor with locks (v2.6)

Edit Metadata on a movie, show or season page (or the pencil on an episode row) edits the title, sort title, original title, year, release date, content rating, critic rating, genres, tagline and summary. Every field you change is **locked**: scans, refreshes, re-enrichment and Fix Match leave it alone. Click a padlock, or *Unlock all*, to hand a field back. An edited content rating applies to rating-limited profiles at once.

In a library, **Select** titles (Shift-click for a range) and choose *Edit metadata…* to set content rating, genres, labels and locks on up to 500 titles in one transaction.

Fix Match and Set Poster

Settings ▸ Library ▸ **Fix Match** lists unmatched items and lets you pick the right match (with a release-year filter); **Set Poster** lists items without artwork. Existing matches are never changed automatically, so use Fix Match on a wrong one.

Labels

Admins can label titles (“kids”, “horror”) from a title's *Labels* box or in bulk with *Add label…*. Labels are visible only to admins and are never changed by metadata refreshes. They drive per-user label restrictions (7.3).

5.9 Collections

- **Franchise collections** are created from TMDB automatically once two films of a series are in the library.
- **Manual collections** (admins, v2.6): *New Collection* on the Collections page; add titles from a title page or a library selection. Order them by hand, release date or title, and choose the cover (a member's poster or

an uploaded image).

- **Smart collections**: rules (movies and/or shows, genre, year range, minimum rating, libraries), refreshed when saved and hourly.
- **Show on Home** puts a collection on everyone's home screen as a row; each person sees only titles they are allowed to see.

5.10 Library health and maintenance

Settings ▸ Library ▸ **Library Health** shows open problem reports from users, integrity-probe results, and unmatched and missing-art counts, with one-click Radarr/Sonarr re-grab. **Maintenance** can re-probe technical metadata (codecs, streams) for files.

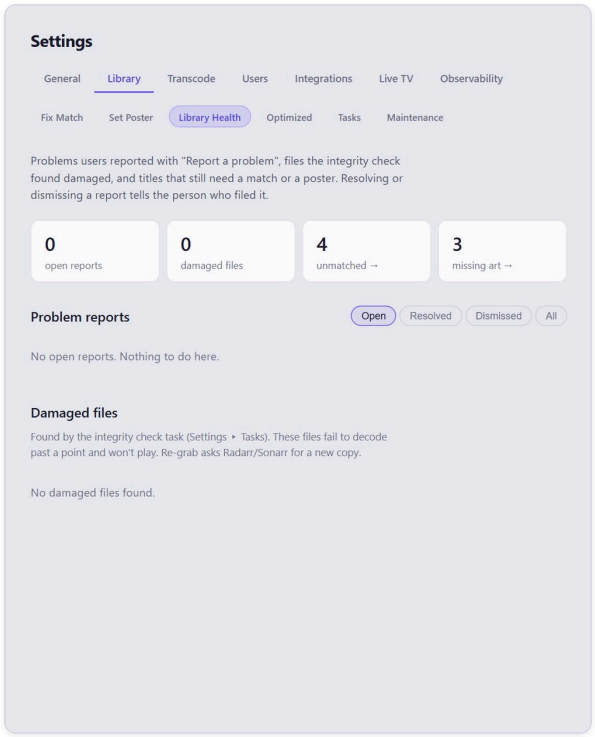


Figure 5-4. Settings ▸ Library ▸ Library Health.

The opt-in *Video integrity probe* task spot-decodes files to find damaged or fake releases before playback does. A file it marks damaged is refused by every app (`FILE_DAMAGED` , 422); replace it and rescan.

6 Transcoding and playback

6.1 How the play decision works

Before playing a file, every app asks the server how to play it (`POST /items/{id}/playback-decision`), sending a description of what the device can decode. The server answers with one of:

Decision	What happens	Server cost
Direct play	The device plays the original file.	File serving only.
Remux (direct stream)	Video is copied into HLS segments; audio is converted only if needed.	Low.
Transcode	Video is re-encoded (codec, bit depth, HDR, resolution or bitrate).	High; uses an encoder slot.

NEW IN v2.6

The server's decision is final. Every v2.6 app plays the answer and decides on its own only when it gets none. The streaming routes enforce it: `GET /media/stream/{id}` refuses a video the server would convert for that viewer (over the internet limit from outside, or over their quality profile) with 409 `TRANSCODE_REQUIRED` , and a damaged file with 422. Music, audiobooks, books and downloads are never refused. Older app builds that decide on their own, and casting (which sends the original file), lose the original for those viewers.

Now Playing (Analytics) shows each stream's decision and the reasons for a transcode.

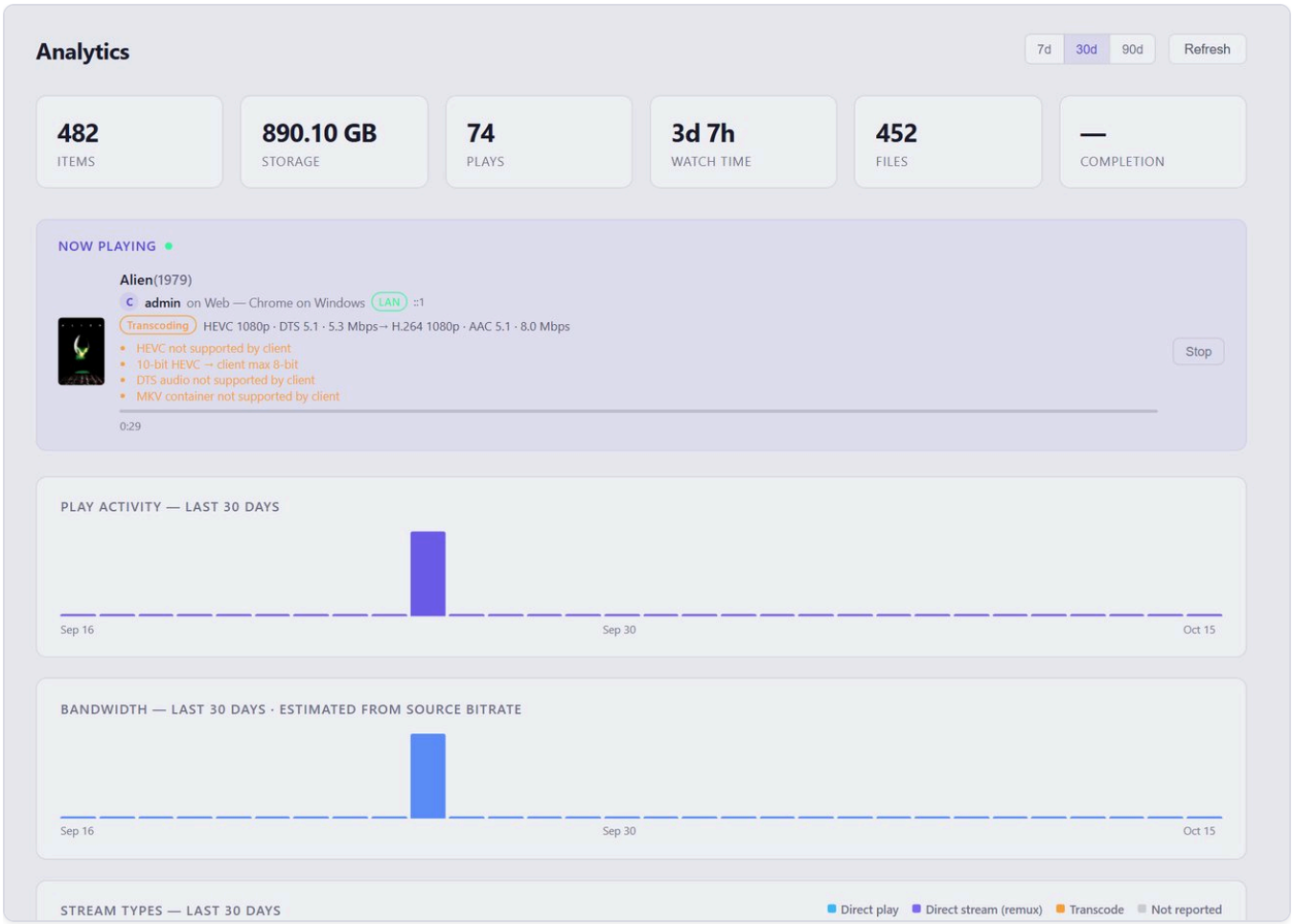


Figure 6-1. Analytics: Now Playing shows the user, device, LAN/remote, decision, source and output formats, the transcode reasons and a Stop button.

6.2 The Transcode settings page

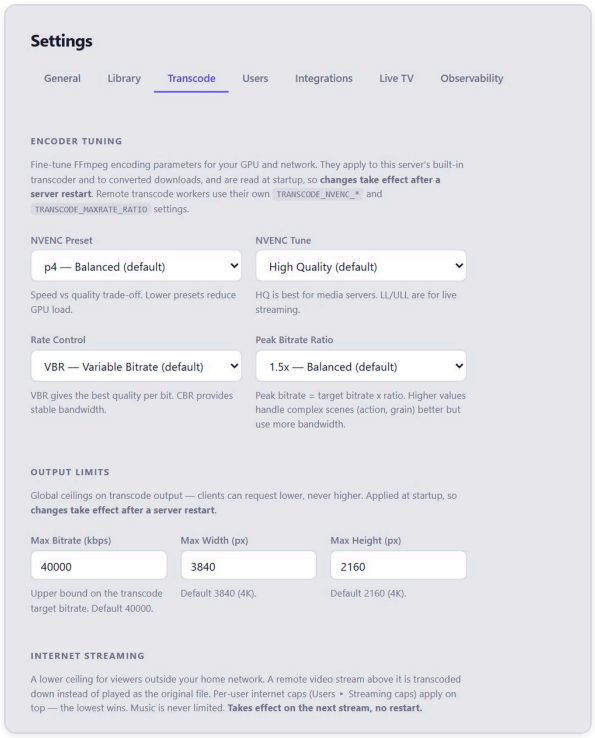


Figure 6-2. Settings ▸ Transcode: encoder tuning, output limits and the internet streaming limit. The page continues with adaptive bitrate and the transcode fleet.

Section	Settings	Takes effect
Encoder Tuning	NVENC preset, tune, rate control; peak bitrate ratio	After a restart. Applies to the built-in transcoder and converted downloads; remote workers use their own `TRANSCODE_NVENC_*` settings.
Output Limits	Max bitrate (kbps), max width, max height	After a restart.
Internet Streaming	Limit internet streams (Mbps); additional local networks	On the next stream, no restart.
Adaptive Bitrate (ABR)	Enable the ladder; hard max rung height; auto max rung height	After a restart.
Transcode Fleet	Embedded worker, local and remote workers: name, encoder, max sessions; worker credentials	Worker overrides are read when the worker starts.

6.3 Encoders and tuning

`TRANSCODE_ENCODERS` (or a worker's encoder in the fleet section) overrides auto-detection with a comma-separated priority list. Accepted tokens: `nvenc`, `qsv`, `vaapi`, `amf`, `software` (H.264), their explicit names (`h264_nvenc`, `libx264`…), the HEVC encoders (`hevc_nvenc`, `hevc_qsv`, `hevc_vaapi`, `hevc_amf`, `libx265` / `hevc_software`) and the AV1 encoders (`av1_nvenc`, `av1_qsv`, `av1_vaapi`, `av1_amf`, `libsvtav1` / `av1_software`). Unknown tokens are ignored; an empty result means software.

- **Encoder failover** (`TRANSCODE_ENCODER_FAILOVER` , on): a job whose hardware encoder can't get the GPU moves to the next encoder on the same machine (NVIDIA → QSV → software).
- **Full-GPU-memory paths** for Intel QSV and VAAPI are on by default and fall back to software decode per job; set `TRANSCODE_QSV_VRAM=false` / `TRANSCODE_VAAPI_VRAM=false` to force software.
- **QSV HEVC decode** into system memory is opt-in per worker (`TRANSCODE_QSV_DECODE`).
- **Peak bitrate ratio**: `maxrate = bitrate × ratio` ; 1.0 caps bandwidth tightly, 2.0 suits fast LANs.

To check which encoder a session used, look for the `starting ffmpeg` log line; its `encoder` field reports what was invoked.

6.4 HDR tone mapping

HDR is tone-mapped to SDR for devices that can't show it. The worker picks the best filter available:

1. `tonemap_cuda` (NVIDIA): scale and tone map entirely in GPU memory, no Vulkan needed.
2. `libplacebo` (Vulkan, any vendor): needs the `graphics` capability in Docker.
3. `scale_cuda` + `zscale` (NVIDIA): GPU downscale, CPU tone map.
4. `tonemap_vaapi` (VAAPI encoders), where its startup probe passes.
5. `zscale` + `tonemap` on the CPU.

The player shows a notice when tone mapping is active.

6.5 The worker fleet

- The **embedded worker** runs inside the server (its segment server binds loopback). Disable it with `DISABLE_EMBEDDED_WORKER=true` or in the fleet section when only standalone workers should transcode.
- **Standalone workers** (`worker` binary or `/usr/local/bin/worker` in the image, the Windows installer's Worker mode) register themselves when started. They need the server's `DATABASE_URL` , `VALKEY_URL` and the same `SECRET_KEY` , and a `WORKER_ADDR` the server can reach.
- The Transcode page can reveal a worker's credentials (the real `DATABASE_URL` and `SECRET_KEY`) after you re-enter your password and 2FA code; the reveal is audited. Serve the API over TLS wherever this is reachable.
- **Storage-less workers**: a worker that can't see the file at its library path pulls the source from the server over HTTP (`/media/stream/{file_id}` with a 24-hour stream token), so no shared NFS/SMB mount is required. Multi-part movies are pulled part by part.
- **Dispatch** is cost-weighted and capability-aware: a 4K job weighs about four 1080p jobs, HDR jobs go to nodes that can tone map on the GPU, AV1 output to AV1-capable nodes, and burn-in and passthrough jobs to workers that advertise those features.

> **WARNING**
> If the embedded worker is disabled and no standalone worker is running, remux and transcode sessions have nowhere to run. Direct play still works.

6.6 Limits: quality, bitrate and streams

Limit	Where	Effect
Output limits	Settings ▸ Transcode	Ceiling on every transcode's bitrate and resolution.
Internet streaming limit (v2.6)	Settings ▸ Transcode ▸ Internet Streaming (off by default)	A remote video over the limit is transcoded down (720p at 4 Mbps, 1080p from 8 Mbps) instead of played as the original; the ABR ladder never exceeds it. Music, audiobooks and podcasts are never limited.
Per-user bitrate cap and internet cap	Users ▸ a user's menu ▸ Streaming caps	A direct play above the cap is re-encoded down; transcodes are capped. A remote stream gets the lowest of the user's cap, their internet cap and the server limit.
Concurrent-stream limit	Users ▸ Streaming caps	Since v2.6 counts direct plays too. Over the limit, playback is refused with 429 `TOO_MANY_SESSIONS` ("Too many streams"). Moving to the next episode or taking a stream over from another device is never refused.
Quality profile (applied since v2.6)	Each user: Account ▸ Playback quality (and in the TV and phone apps)	Maximum resolution, video and audio bitrate, and H.264 or HEVC for conversions. Anything above it is converted, at home too.
Watch limits	Users ▸ Watch limits	Daily minutes and allowed hours (an end earlier than the start makes an overnight window).

The adaptive-bitrate ladder (off by default) serves several renditions the player switches between; *Auto max rung height* (1080 by default) keeps automatic playback from climbing into a 4K rung the device can't decode.

6.7 Optimized versions and converted downloads

Both are encoded on the server by the same runner, with its configured H.264 encoder (hardware where Settings ▸ Transcode allows, else software), `DOWNLOAD_MAX_CONVERSIONS` at a time (1 by default, 2 at most). Each conversion holds one of the embedded worker's transcode slots, so conversions wait while playback is using every slot; users' downloads go before optimized versions.

Optimized versions (v2.6)

Admins choose **Optimize…** on a title, season, show, library or selection, with a profile: *Mobile* (720p, 4 Mbps), *TV* (1080p, 8 Mbps) or *Original quality, compatible* (source resolution, H.264 + AAC). When a device would otherwise need a full transcode, the server can hand it a fitting optimized version to direct-play instead (for apps that declare support; the web player does). Copies live in `CACHE_PATH/optimized`, never in library folders, and don't expire: they are deleted with the original file or by hand. Settings ▸ Library ▸ **Optimized** lists them all.

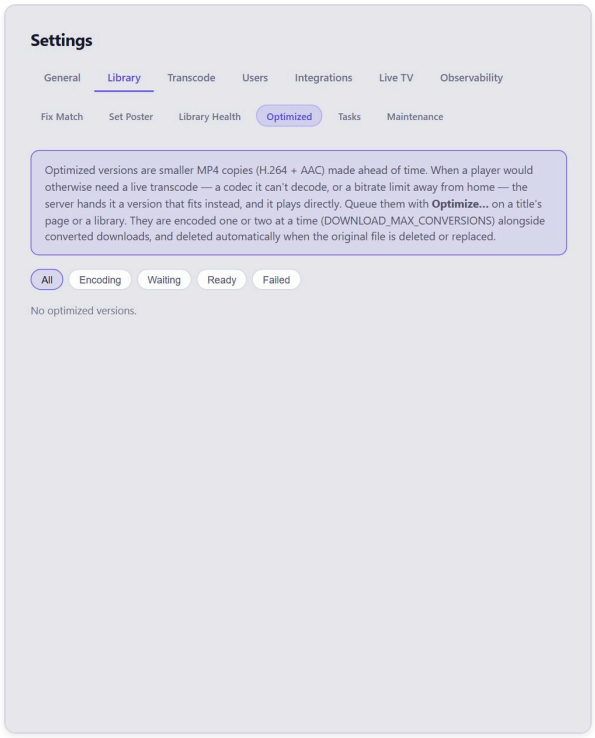


Figure 6-3. Settings ▸ Library ▸ Optimized.

Converted downloads (v2.6)

The Download button offers the original file or a smaller MP4 (1080p 8 Mbps, 720p 4 Mbps, 720p 2 Mbps, 480p 1 Mbps). A file is converted once per quality and shared. Users' streaming caps (and, from outside, the internet limit) bound the qualities offered. Finished files live in `CACHE_PATH/downloads`, kept for `DOWNLOAD_CACHE_TTL_HOURS` after last use and capped by `DOWNLOAD_CACHE_MAX_GB`. Downloads of any kind require **Allow web downloads** (Settings ▸ General ▸ Sharing).

6.8 Subtitles

- **Text subtitles** (embedded or next to the file) are rendered by the players.
- **OCR**: picture subtitles (PGS, VobSub, DVB) can be converted to text with FFmpeg and Tesseract, per stream from the player or for a whole library with the *OCR image subtitles* task (off by default; weekly once enabled). The Docker image ships Tesseract with English, Spanish, French, German, Italian, Portuguese, Japanese, Chinese (simplified and traditional), Korean, Russian and Arabic data.
- **OpenSubtitles** search in the player, and **automatic download** (v2.6, off by default): Settings ▸ General ▸ Automatic subtitle download, with languages (e.g. `en,es`) and a hearing-impaired preference; libraries can override it. The *Download missing subtitles* task runs every 6 hours and only fetches a language the file doesn't already have.
- **Timing for everyone** (v2.6): an admin can correct a subtitle's sync in the web player and choose *Save for everyone*; the offset is applied on the server for every app.

NEW IN v2.6

Image-subtitle burn-in. When a viewer picks a PGS or VobSub track on a server stream, they get its OCR'd text version if one exists; otherwise the subtitle is **burned into the picture** during a full re-encode. Admins don't need to enable anything.

- A burn-in is always a full re-encode on a single rendition. The overlay runs on the CPU, so the session doesn't use the all-GPU-memory decode/scale/tone-map paths and counts as a heavier job. Output is 8-bit. The subtitle's timing offset applies.
- Only workers that advertise the `burn_in_subtitles` feature get burn-in jobs. If none does (every worker is an older build), the start fails with 503 `BURN_IN_UNAVAILABLE` : update the workers. A multi-part movie can't be burned (`BURN_IN_UNSUPPORTED`).
- Refusals happen before the viewer's current stream is touched, so what they're watching keeps playing.

6.9 Audio passthrough on remux

NEW IN v2.6

Apps that bitstream audio to an AV receiver can declare it, and the server then keeps that audio intact on a remux instead of converting it:

- For the declared codecs (AC-3, E-AC-3, TrueHD, DTS) the client's channel limit is ignored, so a 7.1 TrueHD or DTS-HD track plays as-is instead of being downmixed to 5.1 AAC.
- A remux copies the track. TrueHD and DTS are packaged in fMP4 segments, since MPEG-TS can't carry them; this needs a worker that advertises the `audio_passthrough` feature, otherwise the track is converted to AAC as before.
- Full transcodes still output AAC. Apps that don't declare passthrough are unaffected. Track labels such as “DTS-HD MA” or “Dolby TrueHD + Dolby Atmos” appear for files scanned or re-probed from now on.

6.10 Dolby Vision

The scanner records the Dolby Vision profile separately and judges each file by its base layer:

Stream	Plays as
Profile 7 / 8.1 (HDR10 base layer)	HDR10
Profile 8.4 (HLG base layer)	HLG
Profile 8 / 9 record over SDR video	SDR
Profile 5 (no compatible base layer)	Refused by every app and by Optimize

Playing the Dolby Vision layer itself on capable devices is not supported yet. Files an older version tagged as Dolby Vision are re-read once by the next scan.

7 Users and security

7.1 Users and invites

Settings ▸ Users ▸ **Users** lists accounts with their role and request settings. **Create User** makes an account directly; **Invite User** creates a link (valid for 7 days) the person uses to set up their own account. Each user's "…" menu holds library access, watch limits, labels, streaming caps, signed-in devices, password reset and deletion.

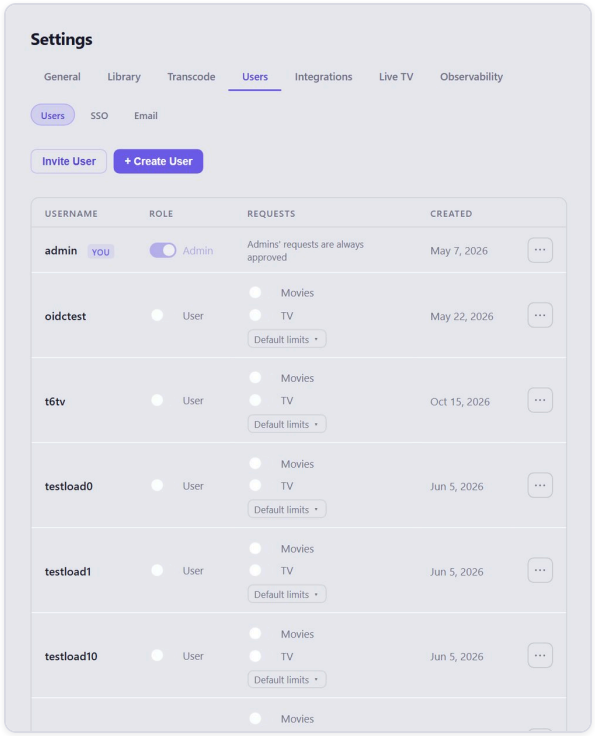


Figure 7-1. Settings ▸ Users: accounts, roles and per-user request settings (test accounts on a development server).

New users settings apply to accounts created from then on (for example request auto-approve; household profiles start with it off). **Request limits** set default request quotas; over quota, a request still goes through but waits for approval. Admins are exempt.

Sign-ins last at most 90 days; after that each device signs in again (TVs re-pair).

7.2 Managed profiles, PINs and rating limits

- Each account can have up to six **managed profiles** (the *Profiles* page in the sidebar), each with its own watch state, favorites and language preferences, an optional 4-digit **PIN**, and a **Max rating**.
- A PIN lets someone switch to that profile without a password. PIN switching can never enter an admin account.
- The **content-rating limit** is enforced everywhere on-demand content appears: browsing, search, the home screen, collections, playback, photos and their EXIF, lyrics, subtitles. Unrated items count as most restrictive. Live TV channels carry no rating and are not covered.

- A rating-limited profile's media requests always need approval.

7.3 Per-user restrictions

- **Library access**: which libraries a user can open. A private library is visible only to users granted it.
- **Labels…** (v2.6): “Never show titles labelled” hides titles with any of those labels; “Only show titles labelled” hides everything else (unlabelled titles too). A label on a show covers its episodes; on an artist or album, its tracks. Managed profiles inherit the restriction; saving signs them out so it applies at once. A hidden title behaves exactly like one above the rating limit.
- **Watch limits**: daily minutes and allowed hours.
- **Streaming caps**: bitrate, internet bitrate and concurrent streams (6.6).
- **View as user** shows the server through a user's restrictions.

7.4 Signed-in devices

Settings ▸ General ▸ Security (admins) or the Account page (everyone) lists each signed-in browser or app with its address, sign-in time and last use, with **Sign out** per device and **Sign out everywhere else**. Admins can see and sign out any user's devices from the Users page. Signing out one device doesn't affect the others.

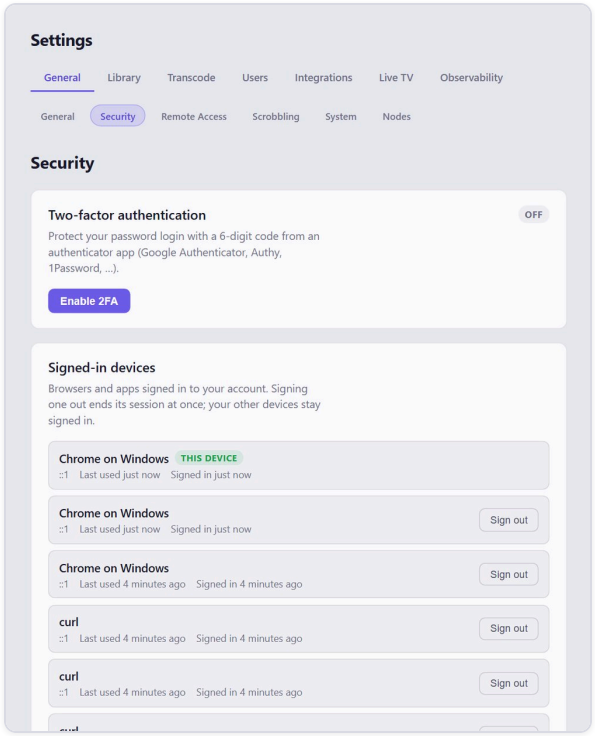


Figure 7-2. Settings ▸ General ▸ Security: two-factor authentication and signed-in devices. The page continues with API keys and the HTTPS certificate.

7.5 Two-factor authentication

Local accounts can enable TOTP two-factor sign-in (**Enable 2FA** on the Security page, or on **Account** for every user) with any authenticator app; recovery codes are issued at enrolment. Every app supports the

second step at sign-in. Codes are single-use within their 30-second window. Sensitive admin actions (revealing worker credentials, restoring a backup) ask for the password and code again. Every user can also change their own password on the Account page; that signs out their other devices.

7.6 Single sign-on (OIDC, SAML, LDAP)

Configure SSO under Settings ▸ Users ▸ **SSO**. There are no environment variables for it. Set the **Public URL** first (and restart) so the URLs OnScreen generates use your public host name.

- **OpenID Connect** (Authentik, Keycloak, Auth0, Azure AD…): redirect URI `https://media.example.com/api/v1/auth/oidc/callback`.
- **SAML 2.0**: SP metadata at `https://media.example.com/api/v1/auth/saml/metadata`; point OnScreen at the IdP's metadata URL. Sign-in is bound to the browser that started it.
- **LDAP / Active Directory**: host, bind credentials, StartTLS or LDAPS, with group sync. Leave *skip TLS verify* off in production.

SAML provisions accounts just in time on first sign-in. Changing an integration's endpoint (OIDC issuer, LDAP host) requires re-entering its secret in the same save.

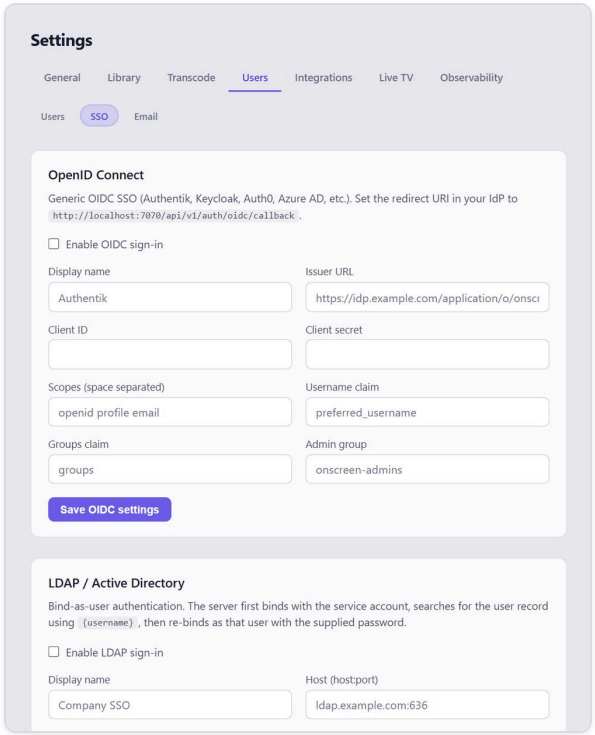


Figure 7-3. Settings ▸ Users ▸ SSO.

7.7 Personal API keys

> **NEW IN v2.6**
>
> Long-lived keys (`osk_…`) for scripts and integrations such as Home Assistant and Kometa.

- Created under Account ▸ API keys (Settings ▸ General ▸ Security for admins), *full* or *read-only*, with an optional expiry. Shown once; the server stores only a hash.
- Sent as `Authorization: Bearer osk_…` or `X-Api-Key: osk_…`, never in a URL.
- A key acts as its owner with their *current* library access, rating limit and labels. Keys are refused for signing in, passwords, PINs, 2FA, pairing, profile switching, device and key management, user administration, data export, server settings and backups (403 `API_KEY_NOT_ALLOWED`). An admin's key can otherwise use the admin API, so give it only to software you trust.
- Admins see every user's keys under *All API keys* and can revoke any; revocation applies on the next request. Creating and revoking are audited.

7.8 Rate limits

Limit	Default
Credential endpoints (sign-in, PIN switch, pairing claims), per address	10/min (`OS_AUTH_RATE_LIMIT_PER_MIN`)
Authenticated API calls, per session	1000/min
Device pairing (code creation and polling)	60/min
Transcode starts	10/min (`OS_TRANSCODE_START_RATE_LIMIT_PER_MIN`)
Discover (TMDB) search; OpenSubtitles search/download	60/min; 30/min
OPDS: wrong keys per address	10 within 15 minutes, then 429

Rate limits are stored in Valkey. If Valkey is unreachable they fail open (counted in `onscreen_ratelimit_failopen_total`) unless `OS_AUTH_RATE_LIMIT_FAIL_CLOSED=true` for the credential endpoints.

7.9 Audit log

Settings ▸ Observability ▸ **Audit Log** records admin, playback and authentication events: settings changes, sign-outs (`auth.session_revoked`), API key creation and deletion, data exports (`user.data_export`), bulk edits, playback stops, optimized-version requests, imports and more. Secrets are never logged.

7.10 Security checklist

- Serve the API over TLS (reverse proxy, certificate files or an uploaded certificate).
- Use a real random `SECRET_KEY`; if it was ever a placeholder, rotate it with `rotate-key`. Use the same key on every worker.
- Keep `METRICS_ADDR` on loopback or firewall port 7071 (pprof is unauthenticated).
- Set a strong `DB_PASS`.
- Behind a reverse proxy, forward `X-Forwarded-For` / `-Proto` on every route and set `TRUSTED_PROXIES` to the proxy.

- Before turning on automatic port mapping: HTTPS, two-factor sign-in for every admin, and a test from outside. Prefer a VPN when you don't need the server open to the internet.
- Use an SMTP relay with TLS (port 465 or STARTTLS): cleartext to a remote relay is refused.
- Disable LAN discovery on untrusted networks; leave LDAP *skip TLS verify* off.
- Allow a notification agent to reach private addresses only for a self-hosted ntfy/Gotify on your LAN.
- Windows: install under `%ProgramFiles%` and keep `.env` admin-only.

> **NOTE**
>
> Known, accepted limits: every shipped deployment connects to PostgreSQL as a superuser; the Windows services run as LocalSystem; the bundled Redis for Windows has no password (it listens on 127.0.0.1 only); installers and binaries are not code-signed. See `docs/security.md` for the full list of residual risks.

8 Live TV and DVR

8.1 Tuners and live broadcasts

Settings ▸ **Live TV** manages channel sources:

- **HDHomeRun**: enter the device's IP or URL; discovery runs immediately.
- **M3U / IPTV**: a playlist URL.
- **Live broadcasts (RTMP)**: create a broadcast, point OBS, ffmpeg or any RTMP encoder at the ingest URL (`rtmp://<host>:1935/live/<key>`) with its stream key, and it appears as a Live TV channel that can be watched and recorded. Set `RTMP_PUBLIC_HOST` if the URL shown should use a different host name.

Each tuner is scanned when added and can be rescanned or disabled. The page also lists unmapped channels and all channels.

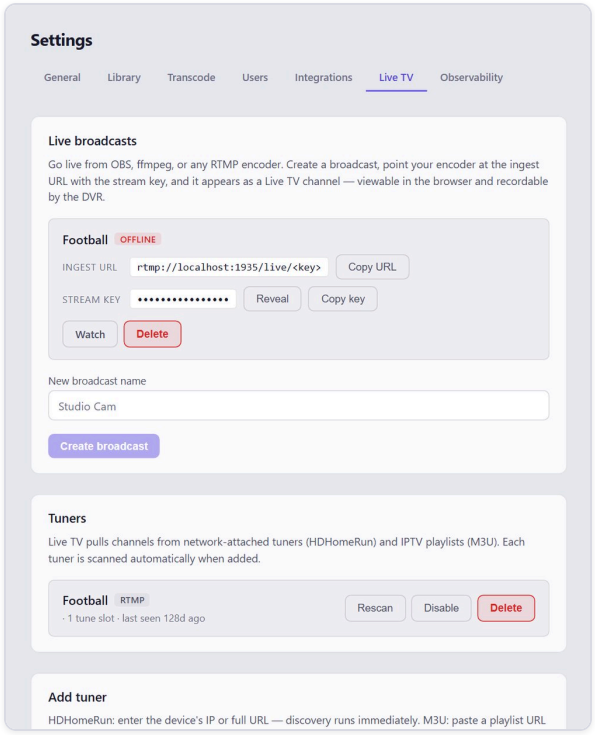


Figure 8-1. Settings ▸ Live TV: live broadcasts, tuners and the add-tuner form.

Built-in limits: at most 6 concurrent live sessions, 8 concurrent DVR captures (3 per user), 100 schedules per user, and 12 hours per recording. Non-admins can't list or tune disabled channels.

8.2 Program guide (EPG)

Add guide data under **EPG sources** as an XMLTV URL or an XMLTV file path. Channels are matched by channel number, then by display name or call sign. The *EPG refresh* task runs every 15 minutes. (Schedules Direct is not offered in the admin UI in v2.6.)

8.3 Recordings and post-processing

Recordings land in a *DVR Recordings* library. The *DVR matcher* task (every minute) turns schedules into recordings, and *DVR retention purge* (daily) removes expired ones.

NEW IN v2.6

Settings ▸ Live TV ▸ **Recording post-processing** decides what happens after a recording finishes. Everything is off by default:

- **Remux** into MKV (or MP4) with clean timestamps for reliable seeking. Fast and lossless; recommended.
- **Transcode to save space**: H.264 or HEVC at high, balanced or small-file quality, using the hardware encoders the Transcode settings allow.
- **Delete original**, only after the new file is checked (length within 2%, video and audio present). Otherwise the original moves to a hidden `.dvr-originals` folder that the scanner skips and retention removes with the recording.
- **Post-processing script**: `ONSCREEN_DVR_POSTPROCESS_SCRIPT` runs after each recording with its path as the only argument, with a 30-minute timeout; output goes to the server log. The settings page shows it read-only.

Recordings are processed one at a time. The Recordings page shows the status; a failure leaves the original untouched, and admins can re-run it from the recording's "⋯" menu.

8.4 Commercial detection

Set **Commercials** in the same panel to *Mark* (a "Skip Ad" button in the web player during breaks) or *Cut* (breaks removed from the file during post-processing; needs remux or transcode).

- With `ONSCREEN_COMSKIP_PATH` (and optionally `ONSCREEN_COMSKIP_INI`) set, OnScreen runs comskip.
- Otherwise a built-in, deliberately conservative detector looks for black frames that coincide with silence: a break must contain at least two ads, last 30 seconds to 6 minutes, and have at least 3 minutes of programme on each side.
- Admins can run *Detect commercials* on any finished recording; this never changes the file.

9 Integrations

Most integrations live under Settings ▸ **Integrations**: Webhooks, Notifications, Arr Services, Plugins, Storage and Import.

9.1 Radarr, Sonarr and requests

OnScreen has a built-in request workflow: users search TMDB (Discover) and request movies or TV seasons; admins approve in a queue, choosing the Radarr/Sonarr instance, quality profile and root folder, or let trusted users auto-approve. In-flight requests show live download status (the *Request download status* task polls every 5 minutes), and an Upcoming calendar shows the arr calendars.

- Add instances under Integrations ▸ **Arr Services**.
- For Radarr/Sonarr to notify OnScreen, set the **Arr Notifications API Key** in Settings ▸ General and use it in the arr's webhook connection.
- Per-user quotas, auto-approve and a can-request switch are on the Users page. Each user can have at most 25 pending requests.

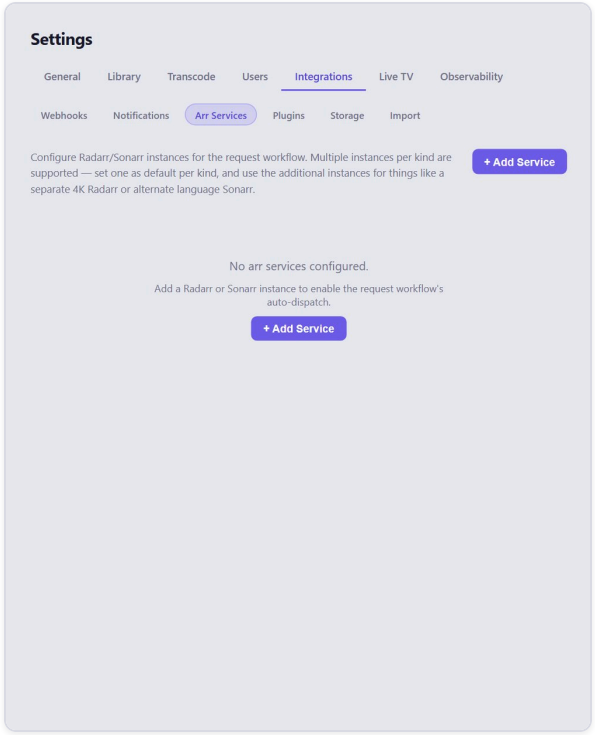


Figure 9-1. Settings ▸ Integrations ▸ Arr Services.

The Android TV, Fire TV, phone and Roku apps contain no request features; requests are made from the web app (and the Tizen and webOS apps).

9.2 Scrobbling

Each user links their own ListenBrainz, Last.fm and Trakt accounts on **Account ▸ Scrobbling** (`/account/scrobble`; the old Settings link redirects there). Last.fm and Trakt links run under the app

credentials entered in Settings ▸ General ▸ *Scrobbling apps*; replacing those credentials later breaks existing links. Imported history (9.7) is not scrobbled.

9.3 OPDS book catalog

Book libraries are published as an OPDS 1.2 catalog at `/opds/` for KOReader, Moon+ Reader, Panels, Chunky, Thorium and others. Users sign in with any user name and a **read-only API key as the password**; account passwords are not accepted there. The catalog follows each user's library access, rating limit and labels, and is open only while **Allow web downloads** is on (otherwise 403 `DOWNLOADS_DISABLED`). Ten wrong keys in a row from one address lock it out for 15 minutes.

9.4 M3U playlists

Playlists can be exported as `.m3u8` (paths relative to the library folder, or full server paths) and imported from `.m3u` / `.m3u8` files up to 1 MB and 10,000 lines. Import matches by path, by relative path, by the same folders on another drive or mount, then by artist and title; it never opens web addresses in the playlist.

9.5 Webhooks and notification agents

- **Webhooks** (Integrations ▸ Webhooks) are signed with HMAC-SHA256 and retried; they are compatible with Overseerr/Tautulli-style receivers. Failed deliveries are counted in `onscreen_webhook_failures_total`.
- **Notification agents** (Integrations ▸ Notifications): Discord, Telegram, ntfy, Gotify and email, for requests, reported problems, new content, failed tasks or backups, and transcode-worker outages. Secrets are encrypted and write-only. Discord and Telegram reach only their official hosts; ntfy/Gotify must use HTTPS unless the agent's *Allow private network* option is on (then plain HTTP is allowed only to private addresses). Email agents use the server's SMTP settings.

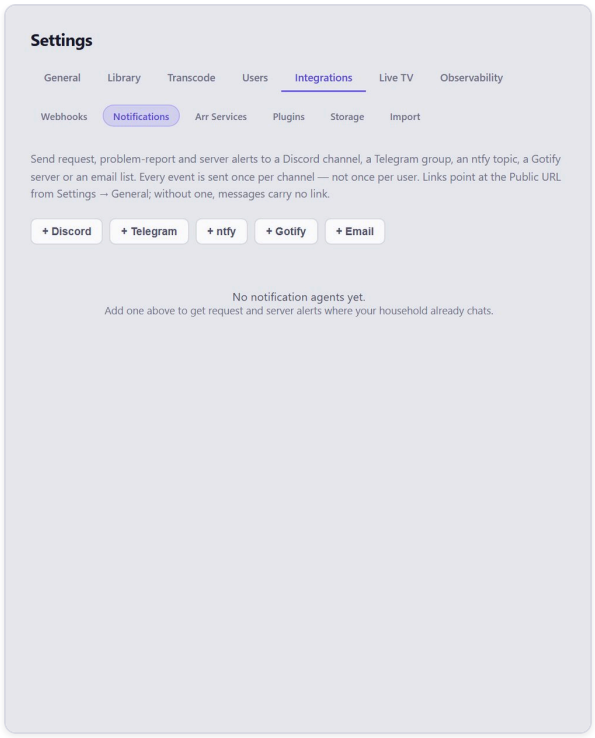


Figure 9-2. Settings ▸ Integrations ▸ Notifications.

9.6 Plugins (MCP)

Plugins are external **MCP servers** that OnScreen calls; OnScreen is the MCP client. Register one under Integrations ▸ **Plugins** with a name, role, endpoint URL and any extra allowed hosts. In this version the only role is `notification`: the plugin must expose an MCP Streamable HTTP endpoint and a `notify` tool, and receives events such as `media.play`, `media.stop`, `media.scrobble` and `library.scan.complete`.

- Plugins must be reachable on a **public** IP; loopback, private and link-local endpoints are refused at dial time. Redirects are not followed.
- Timeouts: 5 s to connect, 10 s per call. Three consecutive failures open a circuit breaker for five minutes; dropped dispatches are audited.

9.7 Import from Plex, Jellyfin or Emby

> **NEW IN v2.6**
>
> Integrations ▸ **Import** brings users' history over from the server you're leaving: watched state, resume points, ratings, favorites, playlists and play history.

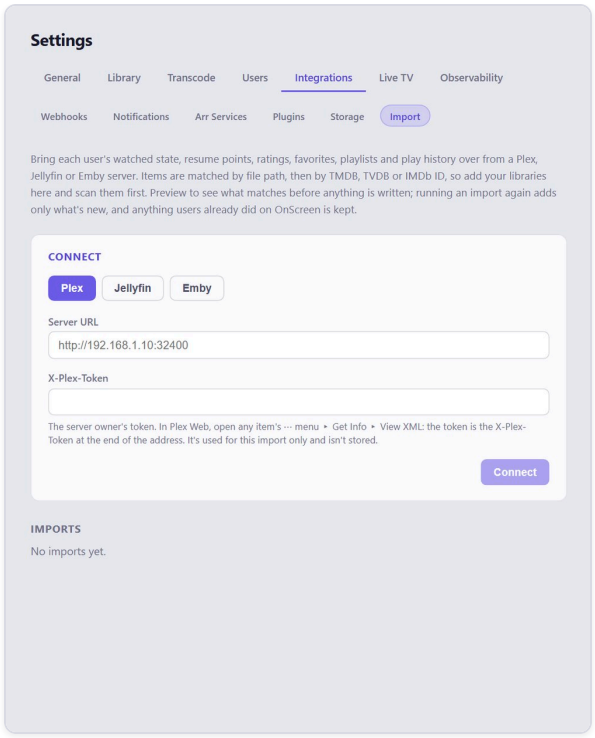


Figure 9-3. Settings ▸ Integrations ▸ Import.

1. Add and scan your libraries in OnScreen first.
2. Connect with the old server's URL and token: for Plex the owner's `X-Plex-Token`; for Jellyfin and Emby an API key. The token is used for the run only and never stored.
3. Map each of its accounts to an OnScreen user (same-name users are suggested) and choose what to bring.
4. **Preview** to see how much matches, then **Import**.

Items are matched by file path first, shows by their folder, then by TMDB, TVDB or IMDb ID. Re-running writes only what's new, and anything a user already did on OnScreen is kept. Plays older than the watch-history retention are left out (the items still count as watched). A Plex server exposes resume points, ratings and playlists only for its owner; Jellyfin and Emby keep no per-play log, so each watched item brings one play. Tested against Jellyfin 12.2 and Emby 4.10; the Plex reader is tested against a model of its API.

9.8 Email and the digest

Settings ▸ Users ▸ **Email** holds the SMTP settings (used for invites, password resets, email notification agents and digests) and a *Send test message* button. Port 465 uses implicit TLS; otherwise STARTTLS is used, and a non-loopback relay that doesn't offer it is refused.

The **digest** (v2.6, off by default) is a weekly or monthly email of what was added, which users opt into on their Account page. The admin turns it on and picks the send day and hour on the Email page; links use the Public URL. Posters are embedded in the email, which carries no access tokens. The *Send digest emails* task checks hourly.

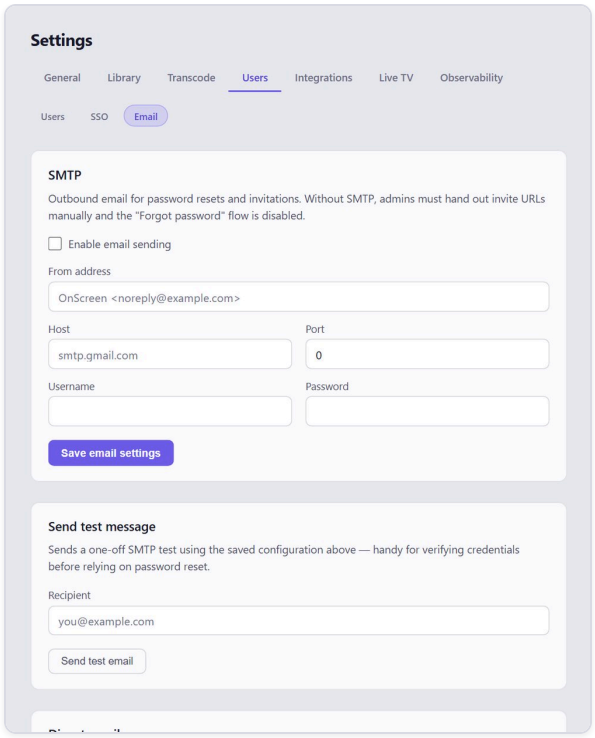


Figure 9-4. Settings ▸ Users ▸ Email.

9.9 Object storage

Integrations ▸ **Storage** switches media storage from local disk to S3-compatible object storage (AWS S3, MinIO, Backblaze B2, Wasabi, Cloudflare R2). Every read and write path then goes through it, and with a CDN base URL set, cacheable bytes are served by signed URLs. There is no environment variable for it. NFO writing is not available with object storage.

10 Tasks and maintenance

10.1 Scheduled tasks

Settings ▸ Library ▸ **Tasks** lists scheduled tasks with their cron schedule, next and last run, an on/off switch, and buttons to run a task now, edit it or delete it. **New Task** adds one (for example a database backup or a library scan). Times are server-local; schedules are standard 5-field cron.

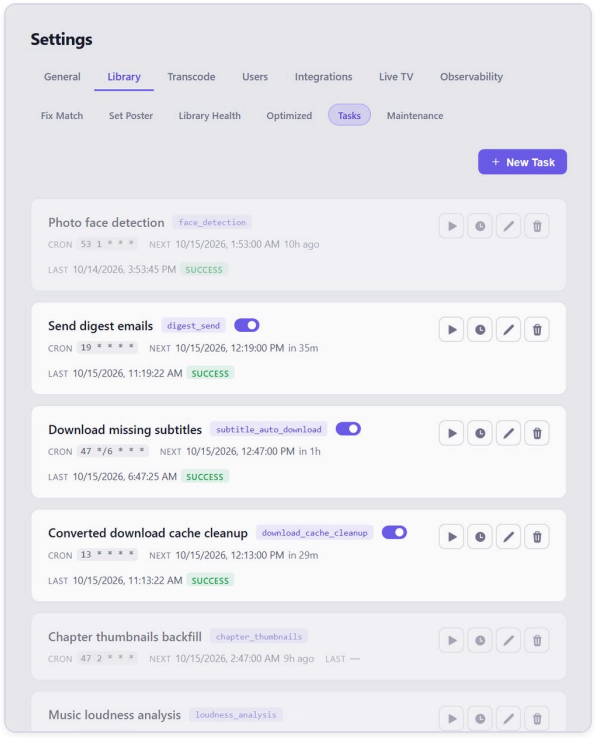


Figure 10-1. Settings ▸ Library ▸ Tasks. Disabled tasks are greyed out.

The server creates these tasks at startup if they are missing:

Task	Schedule	Default
DVR matcher	every minute	on
EPG refresh	every 15 min	on
DVR retention purge	daily 03:17	on
Refresh missing artwork	every 2 h	on
Refresh analytics (watch_plays)	every 10 min	on
Request download status (Radarr/Sonarr)	every 5 min	on
Refresh smart collections	hourly	on
Import NFO collections	daily 03:17	on (does nothing until a library opts in)
Refresh podcast feeds	hourly	on
Send digest emails	hourly	on (does nothing while the digest is off)

Task	Schedule	Default
Refresh IMDb, Rotten Tomatoes and Metacritic scores	hourly	on (needs an OMDb key)
Seek-bar thumbnail backfill	daily 02:29	on
Converted download cache cleanup	hourly	on
Franchise collections	daily 04:53	on
Download missing subtitles	every 6 h	on (does nothing until automatic subtitle download is enabled)
OCR image subtitles	weekly, Sun 04:00	**off**
Video integrity probe	daily 03:41	**off**
Music loudness analysis (v2.6)	daily 04:13	**off**
Photo face detection (v2.6)	daily 01:53	**off**
Chapter thumbnails backfill (v2.6)	daily 02:47	**off**

The heavy sweeps are off by default and work in bounded batches when enabled (for example loudness analysis measures up to 500 files a night; face detection looks at up to 500 photos within 30 minutes). Face data is personal: whether to collect it is your decision, and a photo library's settings can delete all of it. Other task types you can add: `backup_database`, `scan_library`, and `static_abr_preencode` when static ABR is enabled.

10.2 Backup and restore

Replication is not a backup. Keep independent copies of:

- **The database.** Options below.
- `SECRET_KEY`, stored out of band. Without it, encrypted settings in a restored database are unreadable.
- **Media and artwork**, with file-system snapshots or object-storage versioning. The artwork cache (`CACHE_PATH`) can be deleted; it is rebuilt on demand.

From the web UI

Settings ▸ Observability ▸ **Backup**: *Download backup* saves a PostgreSQL custom-format dump whose file name includes the schema version (`-vN.dump`). *Restore database* uploads such a dump and **wipes and replaces** the database; it asks you to type RESTORE and confirm your password (and 2FA code). A dump from an older schema is migrated after the restore; a dump from a newer server is refused (409 `DUMP_NEWER_THAN_SERVER`) unless you override.

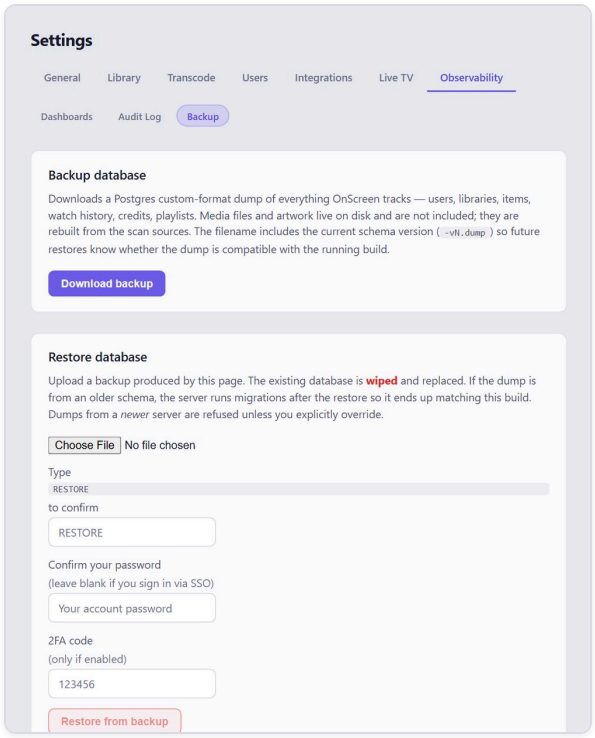


Figure 10-2. Settings ▸ Observability ▸ Backup.

Scheduled backups

Add a `backup_database` task on the Tasks page with an *Output directory* and *Keep last N backups*. It needs `pg_dump` (looked for in `<exe dir>/pgsql/bin`, then `PATH`; the Docker image includes the PostgreSQL 17 client and a `/var/backups/onscreen` directory). Dumps are written owner-only. Mount the output directory on persistent storage.

From the command line

```
pg_dump -U onscreen -Fc onscreen > onscreen_$(date +%Y%m%d).dump
pg_restore -U onscreen -d onscreen --clean onscreen_20261015.dump

# cron, daily at 03:00
0 3 * * * pg_dump -U onscreen -Fc onscreen > /backups/onscreen_$(date +\%Y\%m\%d).dump
```

Test restores periodically.

10.3 Upgrading from v2.5 to v2.6

1. **Back up the database** (and confirm you have `SECRET_KEY`).
2. **Pull the new image** (`ghcr.io/collinjaycock/onscreen:2.6`) or install the new binaries/package. If you built the GPU image, rebuild it.
3. **Apply migrations 00037–00057** (there is no 00053) with your migrate step, or set `AUTO_MIGRATE=true`. `/health/ready` stays unready until they are applied.
4. **Behind a reverse proxy or Cloudflare, set `TRUSTED_PROXIES`**, or the new internet streaming limit never applies (4.3).

5. **Check three settings that used to be ignored** and now apply: each library's metadata language (on the next metadata refresh), users' concurrent-stream limits (direct plays now count) and users' quality profiles (anything above is converted, at home too).
6. **Run a scan** of your libraries. The next scan does one-off work without re-probing files: episodes filed as `E01` whose file is `E01–E02` become multi-episode items, editions and multi-part movies are read from file names, featured-artist credits are read, and photos are regrouped (told apart by file, with RAW + JPEG pairs as one photo).
7. **Optional:** add OMDb and fanart.tv keys (Settings ▸ General); enable the new off-by-default tasks you want (loudness analysis, face detection, chapter-thumbnail backfill) and automatic subtitle download; set the new environment variables if needed (`ONSCREEN_DVR_POSTPROCESS_SCRIPT`, `ONSCREEN_COMSKIP_PATH` / `_INI`, `DOWNLOAD_MAX_CONVERSIONS`, `DOWNLOAD_CACHE_TTL_HOURS`, `DOWNLOAD_CACHE_MAX_GB`).
8. **Update the apps.** The server holds every app to its play decision; older app builds that decide on their own lose the original file for viewers over the internet limit or their quality profile, and casting a video over the viewer's quality profile fails.
9. **Update every standalone worker** to v2.6. Image-subtitle burn-in and TrueHD/DTS passthrough are sent only to workers that advertise them.

Existing show matches are kept even though matching is stricter now; use Fix Match on any that are wrong. The full list is in `CHANGELOG.md` under “Upgrade notes (from v2.5.x)” and in `docs/releases/v2.6.0.md`.

> **NOTE**
> Upgrading from v2.4 or earlier? Go through v2.5 first: it requires `DB_PASS` in Compose, refuses placeholder `SECRET_KEY` values, changes the nginx sample, and limits sign-ins to 90 days. See the v2.4.x → v2.5.0 section of `docs/deployment.md`.

10.4 Monitoring, metrics and logs

Health endpoints

Endpoint	Meaning
`GET /health/live`	The process is up. Use for liveness checks.
`GET /health/ready`	Database and Valkey reachable *and* no pending migrations. The Docker image's health check uses it.
`GET /health/cluster`	`{site_id, role, replication_lag_seconds}` for multi-site setups; 503 when the database is unreachable.
`GET /health/version`	The version only (build details are on the metrics port).
Worker `:7074/health/live`, `/health/ready`	Standalone worker health.

Prometheus metrics

`METRICS_ADDR` (default `127.0.0.1:7071`) serves `/metrics` with Go runtime and process metrics plus:

Metric	Tracks
`onscreen_http_requests_total`, `onscreen_http_request_duration_seconds`	Requests and latency by method, route template and status
`onscreen_db_query_duration_seconds`	Query duration by SQL verb
`onscreen_transcode_sessions_active`, `onscreen_transcode_jobs_total`	Active transcodes; jobs dispatched, queued or failed
`onscreen_scanner_files_scanned_total`	Files scanned per library
`onscreen_watch_events_total`	Watch events by type
`onscreen_webhook_failures_total`	Webhook deliveries that failed after all retries
`onscreen_hub_cache_refresh_duration_seconds`	Home-screen cache refresh time
`onscreen_ratelimit_failopen_total`	Requests let through because Valkey was unavailable

Series with labels appear only after the first matching event.

Tracing, logs and analytics

- **OpenTelemetry tracing** (OTLP/gRPC) is configured in Settings ▸ Observability ▸ Dashboards; restart after changing it. The repository's Compose file has a `tracing` profile that starts Jaeger.
- **Logs** are structured and written to standard output (`docker compose logs -f server`, `journalctl -u onscreen`, or the Windows `logs\` folder). The level is set in Settings ▸ General ▸ Server (restart required). Admins can also read the most recent 2,000 log entries through `GET /api/v1/admin/logs` where there is no shell access.
- **Analytics** (sidebar) shows plays, watch time, bandwidth, stream types, codecs, top titles and users, and Now Playing with an admin **Stop**: it tears down a transcode and, for two minutes, refuses that user's requests for that title from that address.

10.5 High availability (optional)

A standard install is a single node. Every tier below is opt-in:

- **Valkey Sentinel**: deploy `docker/docker-compose.valkey-ha.yml` and set `VALKEY_SENTINEL_ADDRS`.
- **PostgreSQL failover**: `docker/docker-compose.postgres-ha.yml` (primary plus streaming standby) and a multi-host `DATABASE_URL`; automatic promotion needs an orchestrator (managed Multi-AZ, Patroni, repmgr) or a manual `pg_ctl promote`.
- **Several API instances** behind a load balancer; singleton work is leader-elected through Valkey. Settings ▸ General ▸ Nodes holds per-node values.
- **Object storage and a CDN** (9.9), `PUBLIC_ASSET_CACHE`, and static-ABR pre-encoding.
- **Multi-site DR**: `SITE_ID` and `/health/cluster`.

Procedures for each tier, failover and fail-back are in `docs/dr-runbook.md`.

11 Troubleshooting

The server won't start: “SECRET_KEY must be at least 32 bytes” or “looks like a placeholder”

Generate a real key with `openssl rand -hex 32`. If an existing install used a placeholder, rotate it with `rotate-key` (3.3) instead of replacing it, so stored secrets are re-encrypted rather than lost.

The server won't start: “DATABASE_URL is required”

`DATABASE_URL`, `VALKEY_URL` and `SECRET_KEY` must all be set: check `.env`, the Compose file or the systemd `EnvironmentFile`. Setting only one of `TLS_CERT_FILE` / `TLS_KEY_FILE` is also a startup error.

Compose refuses to start: “DB_PASS is required”

Set `DB_PASS` in `.env`. For an existing database volume, use the password it was created with.

The container stays unhealthy, or sign-in fails after an upgrade

Migrations are pending: `/health/ready` stays unready until they are applied. Run the migrate step (3.4) or set `AUTO_MIGRATE=true`.

Database connection refused

- Check PostgreSQL is running: `pg_isready -h localhost -p 5432`.
- Local connections usually need `?sslmode=disable`.
- Inside Compose, services connect by service name (`postgres`, `valkey`), not `localhost`.
- “Remaining connection slots are reserved”: terminate idle connections (the pool reconnects) and give containers a stop grace period of at least 35 seconds so the server can drain.

Migrations fail

Create the `pg_trgm`, `pgcrypto` and `unaccent` extensions as a superuser, check the database user can create schema objects, and run `goose status` to see what was applied.

Files don't appear after a scan

- Check each library path is the path *the server* sees, and that the process can read it. In Docker: `docker compose exec server ls /media`.
- Linux service: the unit runs as the installing user with `ProtectHome=read-only`; make the media readable by that user.
- Look for scan errors: `docker compose logs server | grep -i scan`.

No cover art or metadata

Set a valid TMDB key in Settings ▸ General and check the log for TMDB rate-limit errors (lower the TMDB rate limit in Settings ▸ System). Wrong show matches: use Fix Match.

Remux or transcode fails while direct play works

- Check a worker is available: the embedded worker may be disabled (`DISABLE_EMBEDDED_WORKER` or the fleet section) with no standalone worker running.
- A standalone worker with `WORKER_ADDR=:7073` makes the server proxy to its own loopback, so every playlist fails with 503. Use an address the server can reach.
- Bare metal: confirm `ffmpeg -version` works for the service user (the Linux tarball relies on the distribution's FFmpeg).
- Image subtitles burned in fail with `BURN_IN_UNAVAILABLE` : update your workers to v2.6.
- "Too many streams" (`TOO_MANY_SESSIONS`): the user's concurrent-stream limit, which now counts direct plays.
- Playback refused as damaged (`FILE_DAMAGED`): the integrity probe flagged the file; replace it and rescan.

The GPU isn't used

- Read the worker's `transcode worker ready` line and the `encoders available` log (2.3).
- `libplacebo=false` : `NVIDIA_DRIVER_CAPABILITIES` lacks `graphics` as applied to the container.
- `cuda_scale=false` with `nvdec_hevc=true` : the host driver is older than 570 (CUDA mismatch), or `/var/cache/onscreen` isn't persistent and writable, so the JIT cache never warms. The first transcode after a fresh deploy being slow is that one-time warm-up.
- VAAPI: `vainfo` must show encode entry points and the user must be in the `video` / `render` groups; the hardened systemd unit may need loosening for some drivers.
- NVIDIA without Docker: `ffmpeg -hide_banner -encoders | grep nvenc` must list NVENC; some distribution builds lack it.

Every viewer shows as LAN, or the internet limit never applies

The proxy isn't forwarding the client address, or it isn't in `TRUSTED_PROXIES` (4.3). With nginx, a location-level `proxy_set_header` silently drops the forwarded headers for that location.

First-run setup is refused

Open the server from the local network by IP or a local name (2.7), or set `ALLOW_PUBLIC_SETUP=true` temporarily.

Notifications only appear after a page reload

The proxy is buffering or timing out the event stream at `/api/v1/notifications/stream` : set `proxy_buffering off` and a long `proxy_read_timeout` (nginx) or `flush_interval -1` (Caddy).

A node is unreachable after changing its listen address

Start it with `IGNORE_NODE_DB_CONFIG=true` , fix the value under Settings ▸ General ▸ Nodes, and restart without the flag.

High memory use during scans

Lower the file-scan concurrency in Settings ▸ System (default `cores × 2`) and restart.

The desktop app says “Failed to fetch”

A server older than v2.5 refuses the desktop app's origin. Update the server, or add `http://tauri.localhost, tauri://localhost` to CORS allowed origins and restart.

Email isn't sent

A remote SMTP relay without TLS is refused; use port 465 or a relay that offers STARTTLS. Digests also need the digest turned on in Settings ▸ Users ▸ Email.

12 Appendix

12.1 Ports

Default	Variable	Service
`7070/tcp`	`LISTEN_ADDR`	API, web UI, TV web app (`/tvapp/`), OPDS (`/opds/`)
`127.0.0.1:7071/tcp`	`METRICS_ADDR`	Prometheus metrics, pprof
`1935/tcp`	`RTMP_LISTEN_ADDR`	RTMP ingest
`7073/tcp`	`WORKER_ADDR`	Standalone worker segment server
`7074/tcp`	`WORKER_HEALTH_ADDR`	Standalone worker health
`7368/udp`	`DISCOVERY_PORT`	LAN discovery
`5432` , `6379`	(Compose)	PostgreSQL and Valkey, published to 127.0.0.1 only

12.2 File and directory locations

What	Location
Cache root (Docker images)	`/var/cache/onscreen` (`CACHE_PATH`); GPU image also `CUDA_CACHE_PATH=/var/cache/onscreen/nv`
Cache root (binaries, default)	`~/.onscreen/cache/artwork`
Converted downloads / optimized versions	`CACHE_PATH/downloads` , `CACHE_PATH/optimized`
Backup directory (Docker images)	`/var/backups/onscreen` (mode 0700)
Migrations	Image: `/migrations` ; source: `internal/db/migrations` ; embedded in the server binary
Binaries (image)	`/usr/local/bin/server` , `/usr/local/bin/worker` , `/usr/local/bin/goose`
Windows installer	Program: `C:\Program Files\OnScreen` ; data and logs: `%ProgramData%\OnScreen` (PostgreSQL data in `pgdata`)
Windows zip logs	`logs\onscreen.out.log` , `logs\onscreen.err.log` (10 MB rotation, 8 generations)
Linux tarball	Install directory with `.env` ; systemd unit `/etc/systemd/system/onscreen.service`
`.env` lookup	Working directory, then the executable's directory
Kept DVR originals	Hidden `.dvr-originals` folder beside the recording

12.3 Command-line tools

Tool (`cmd/`)	Purpose
`server`	The OnScreen server. `server migrate` applies pending migrations to `DATABASE_URL` and exits.
`worker`	Standalone transcode worker.
`rotate-key`	Re-encrypts stored secrets from `OLD_SECRET_KEY` to `SECRET_KEY`; dry run unless `-apply`. Flags `-old-key`, `-new-key`, `-database-url` override the environment.
`onscreen-cli`	Terminal client that browses a server and plays through mpv (reads `ONSCREEN_USERNAME` / `ONSCREEN_PASSWORD`).
`devtoken`	Developer tool that mints admin tokens from `SECRET_KEY`. Not shipped in any installer; delete old copies.

Useful `make` targets: `build`, `migrate`, `migrate-status`, `docker`, `installer-windows`, `installer-windows-msi`, `installer-linux`; `make help` lists them all.

12.4 API documentation

- REST reference: `docs/api/openapi.yaml` (all routes under `/api/v1`).
- Error codes: `docs/api/ERROR_CODES.md` (for example `TRANSCODE_REQUIRED`, `TOO_MANY_SESSIONS`, `FILE_DAMAGED`, `UNSUPPORTED_CONTAINER`, `BURN_IN_UNAVAILABLE`).
- Personal API keys: `docs/api/api-keys.md`. OPDS: `docs/opds.md`. Plugins: `docs/plugins.md`.
- Client capability profiles (how apps describe what they can play): `docs/capability-profiles.md`.
- Deployment details, HA and security: `docs/deployment.md`, `docs/dr-runbook.md`, `docs/security.md`, `docs/key-rotation.md`.